

DDD SAMPLE TUTORIAL

A SAMPLE DDD TUTORIAL IS GIVEN HERE TO SHOW YOU SOME OF THE MAIN FEATURES IN DDD AND SHOW YOU HOW TO USE DDD TO EXECUTE INSTRUCTIONS IN SINGLE STEP MODE (EXECUTING ONE INSTRUCTION AT A TIME) AND TO VIEW THE REGISTERS AND MEMORY CONTENTS.

PROGRAM USED IN THIS TUTORIAL IS `sample1.as` WHICH IS GIVEN BELOW

`sample1.as`

```
.global _start

.data
list: .space 40

.text
_start: novq $0x9876abcdef543210, %rax
        novq $list, %rsi
        novq %rax, (%rsi)
        novq (%rsi), %rdx
        addq %rax, %rdx
        pushq %rax
        pushq %rdx
        popq %rax
        popq %rdx

        novq $60, %rax
        xorq %rdi, %rdi
        syscall
```

ASSEMBLE, LINK AND CREATE EXECUTABLE OBJECT FILE

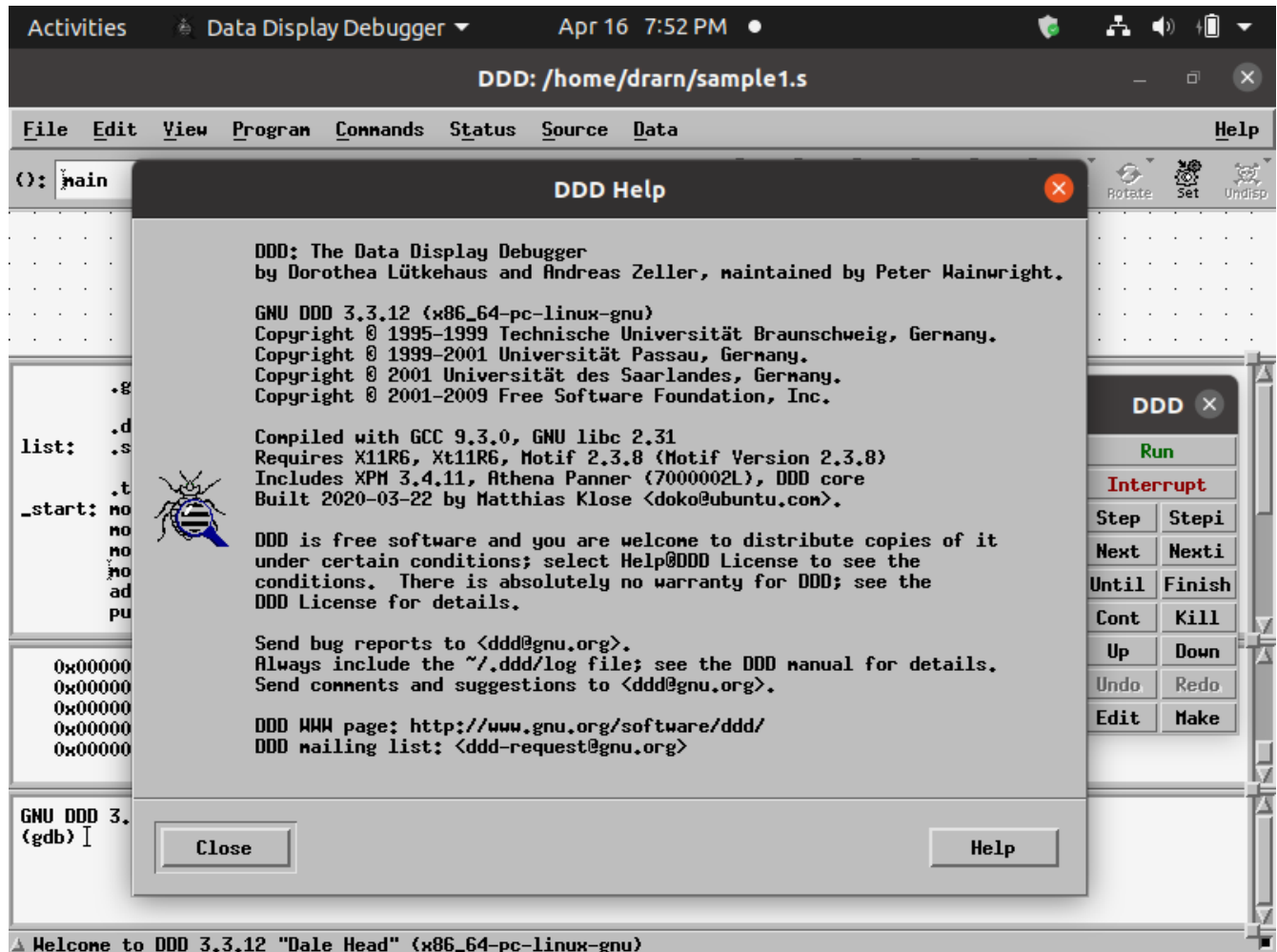
`$as -gstabs -o sample1.o sample1.s`

`$ld -o sample1 sample1.o`

INVOKE DDD DEBUGGER USING THE FOLLOWING COMMAND
TO DEBUG sample1

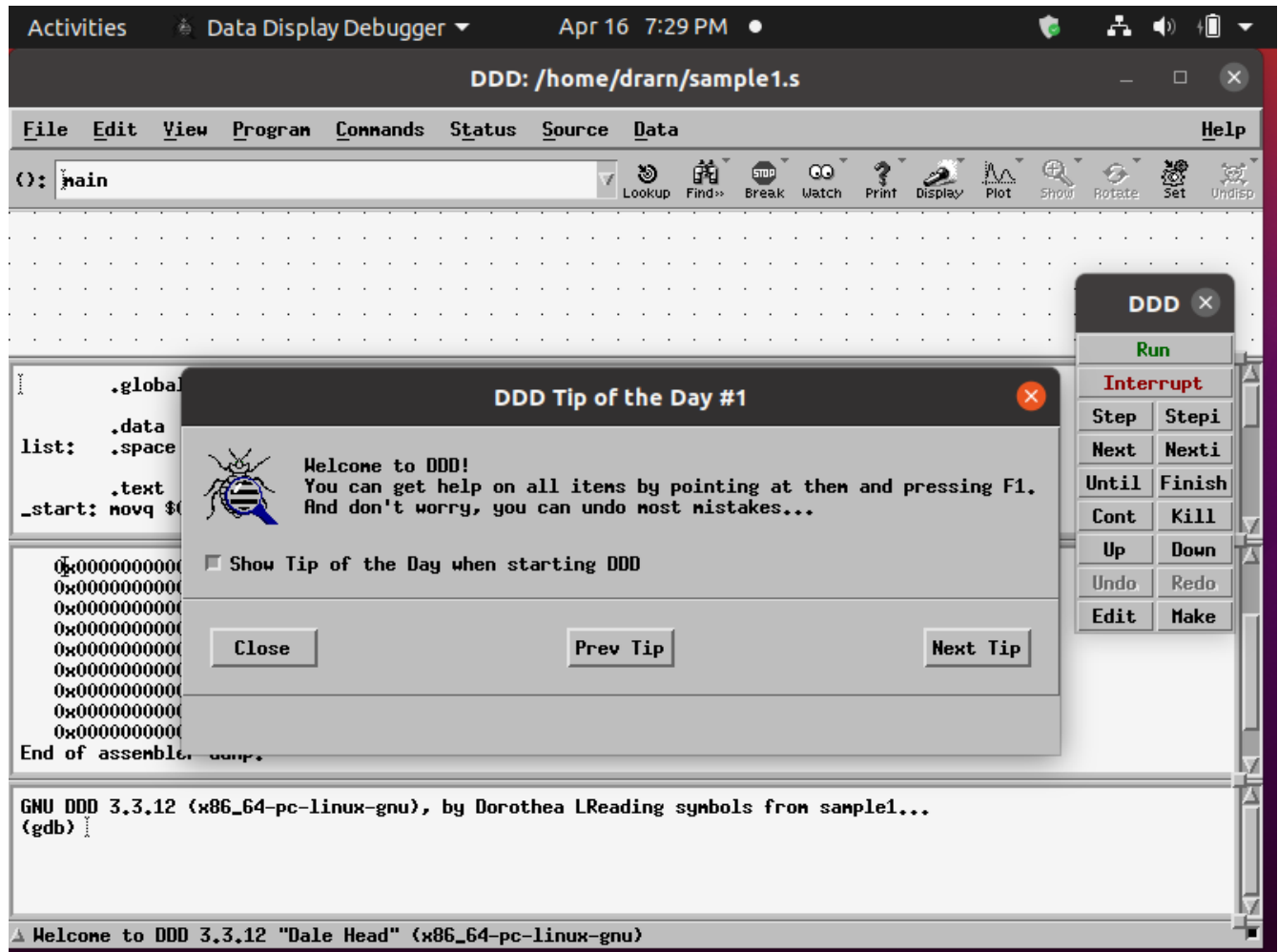
\$ddd sample1&

DDD DEBUGGER OPENING WINDOW SHOWING DDD Help



CLOSE DDD HELP

DDD DEBUGGER WINDOW SHOWING DDD Tip of the Day #1



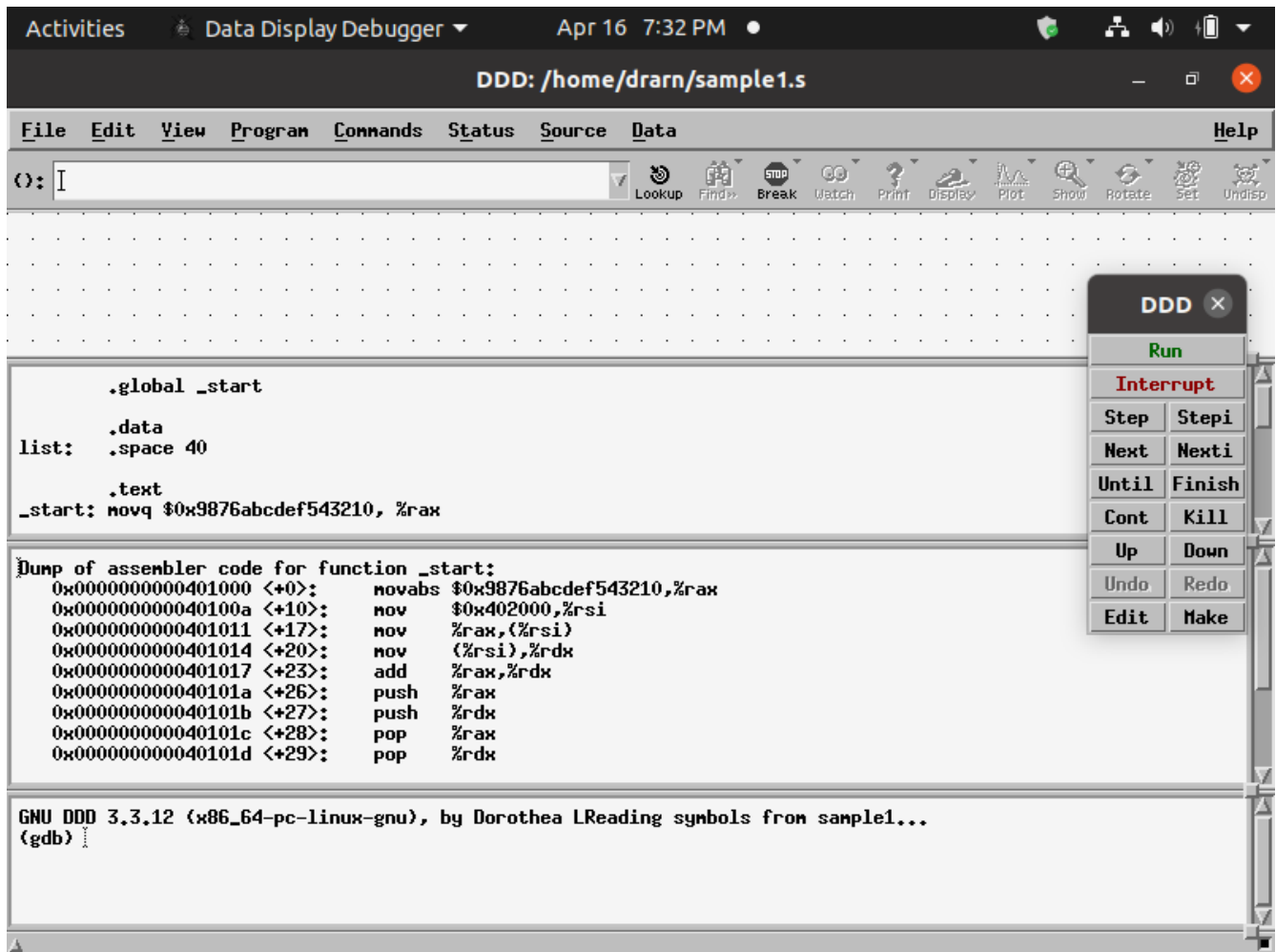
YOU CAN GO THROUGH ALL THE TIPS BY CLICKING NEXT TIP BUTTON

Close DDD Tip of the Day

NOW LOOK AT THE DDD DEBUGGER MENU OPTIONS

File Edit View Program Commands Status Source Data

CLICK ON EACH MENU OPTION TO KNOW THE SUB OPTIONS UNDER EACH MENU OPTION



ALSO LOOK AT THE DDD DEBUGGER SCREEN WINDOWS

DATA WINDOW

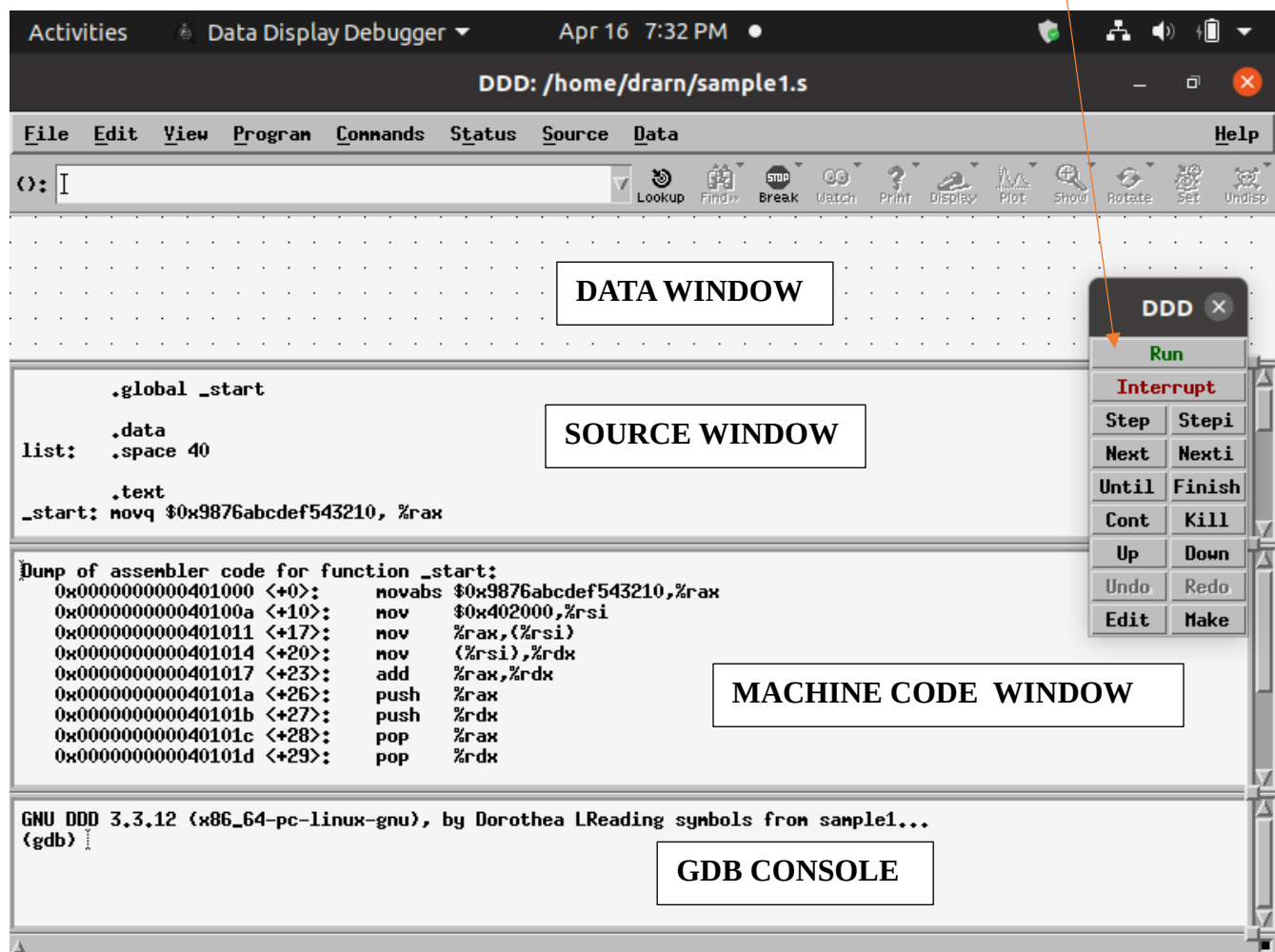
SOURCE WINDOW

MACHINE CODE WINDOW

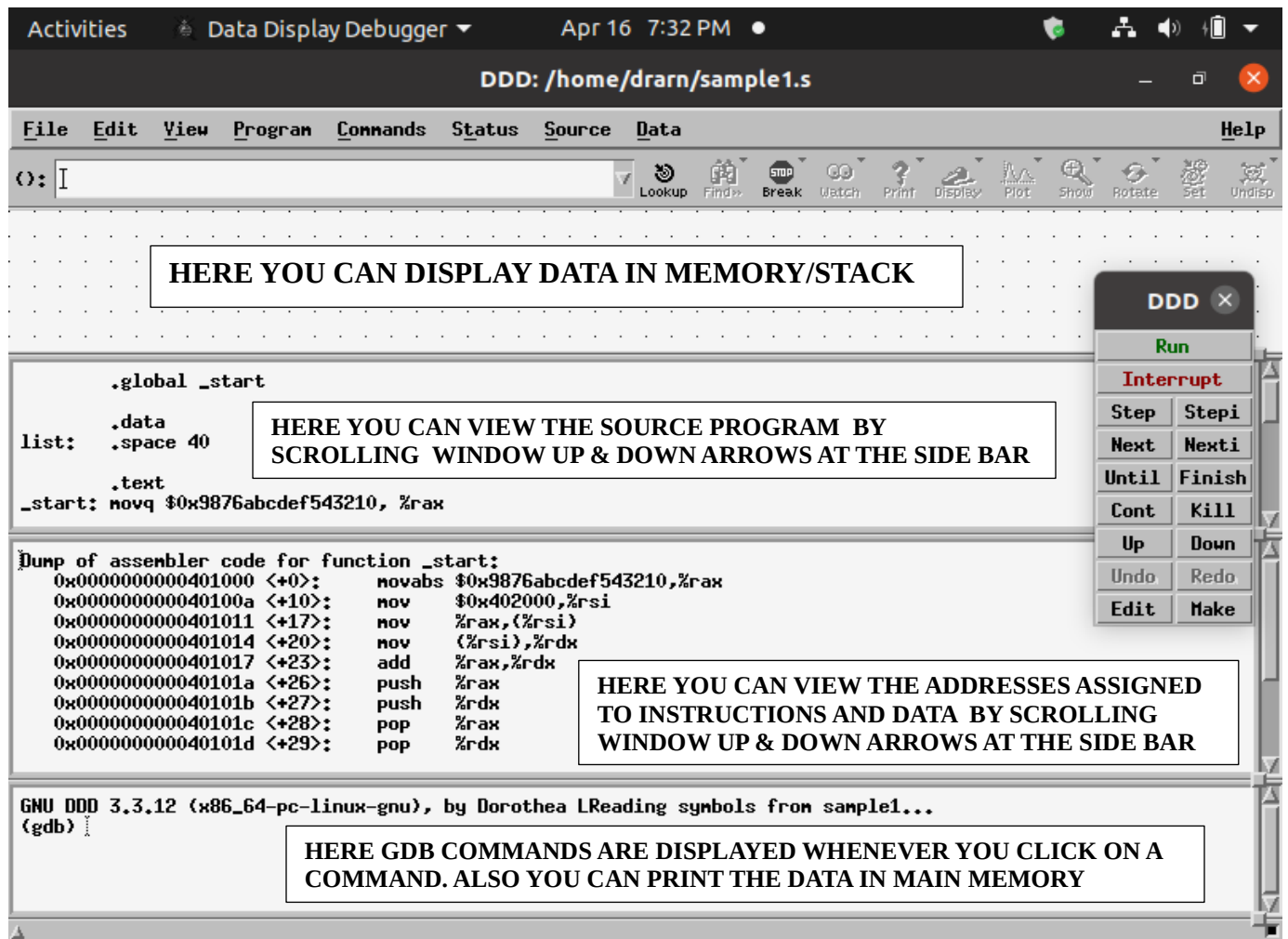
GDB CONSOLE

COMMAND WINDOW ON TOP

COMMAND WINDOW



DDD DEBUGGER WINDOWS

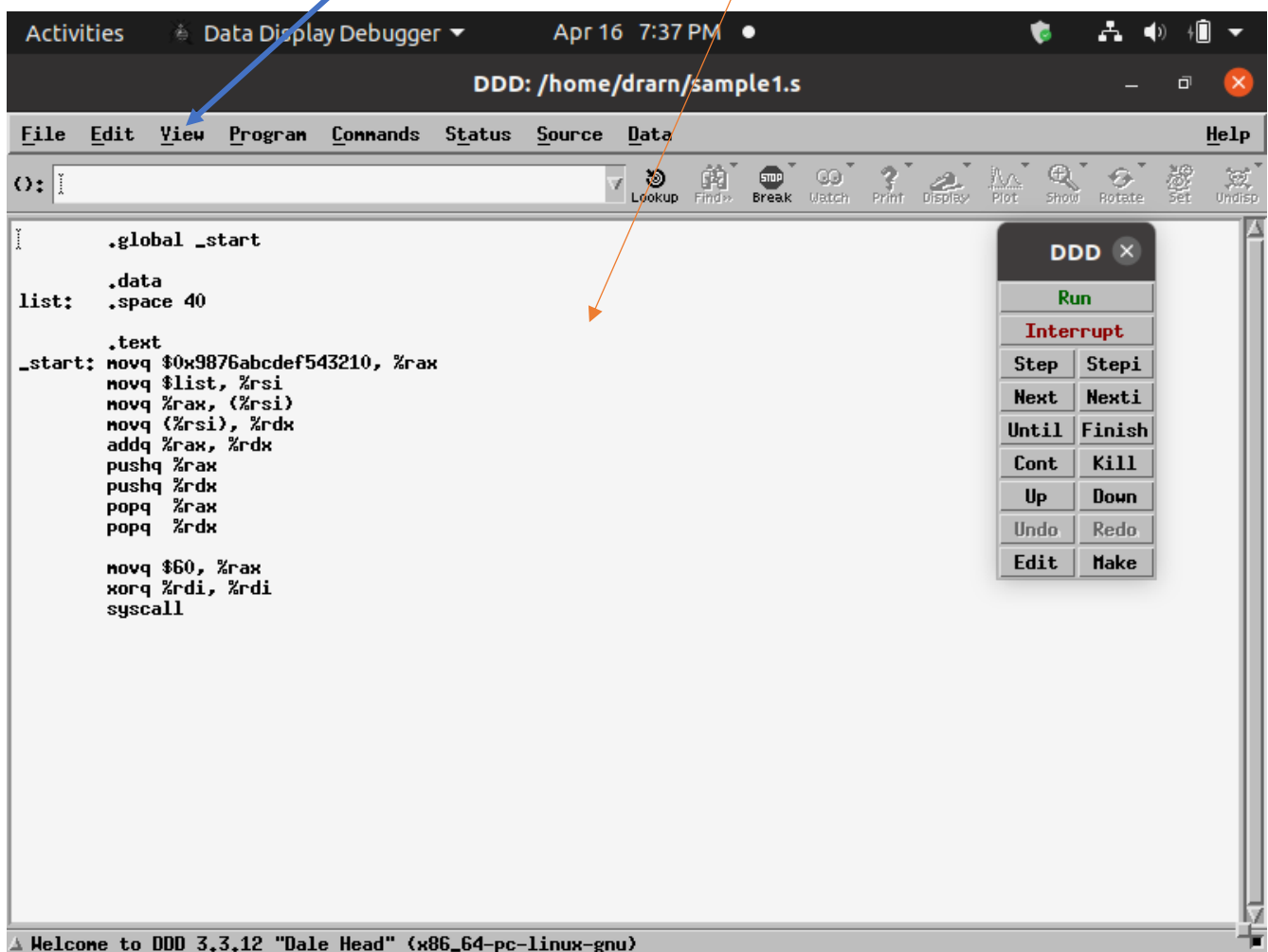


TO ENABLE OR DISABLE WINDOWS, CLICK ON **View** MENU OPTION TO SEE THE WINDOWS OPTIONS

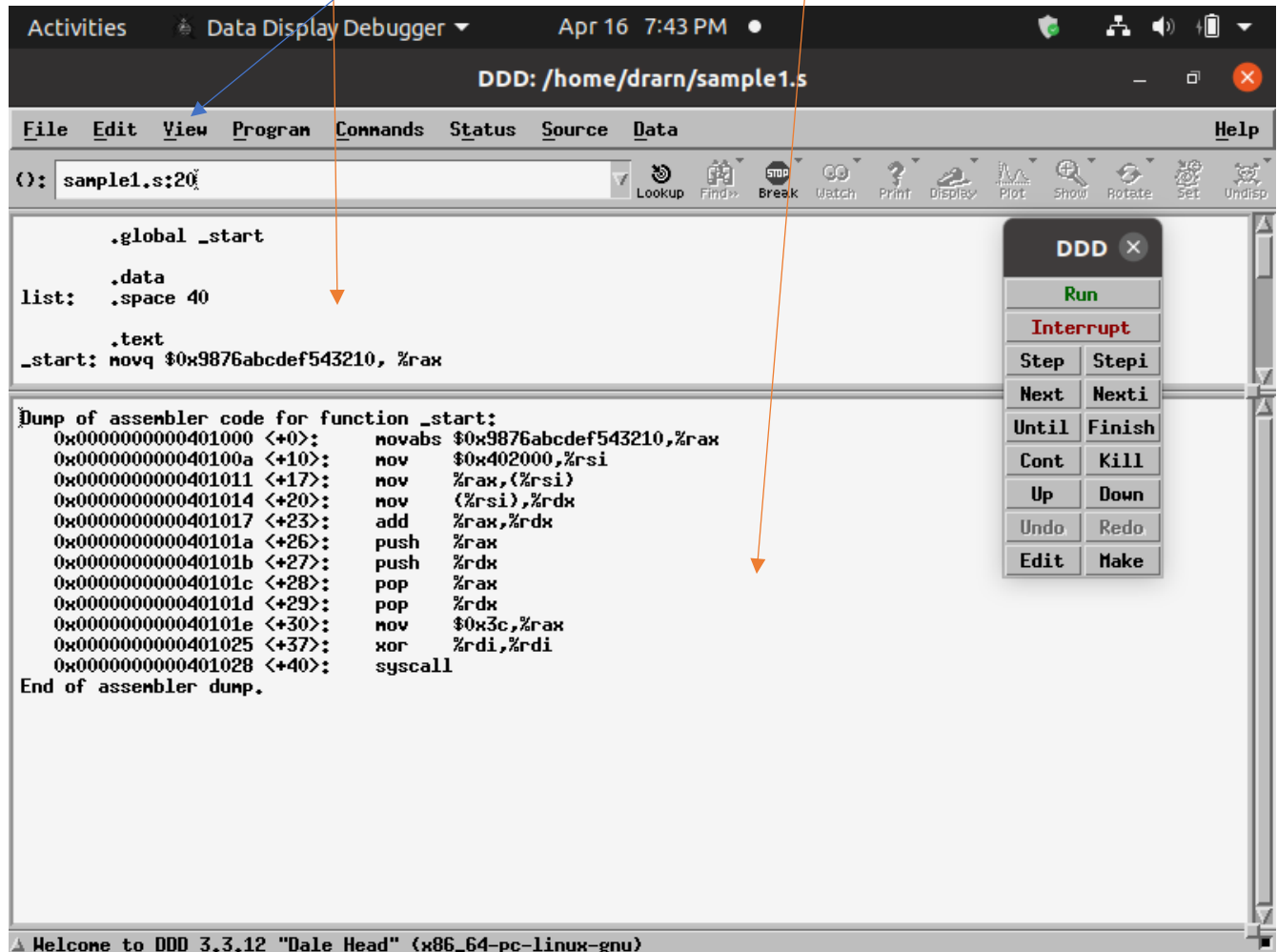
- ❖ DATA WINDOW
- ❖ SOURCE WINDOW
- ❖ MACHINE CODE WINDOW
- ❖ GDB CONSOLE

BY CLICKING ON THESE OPTIONS YOU CAN ENABLE OR DISABLE SELECTIVELY DIFFERENT WINDOWS

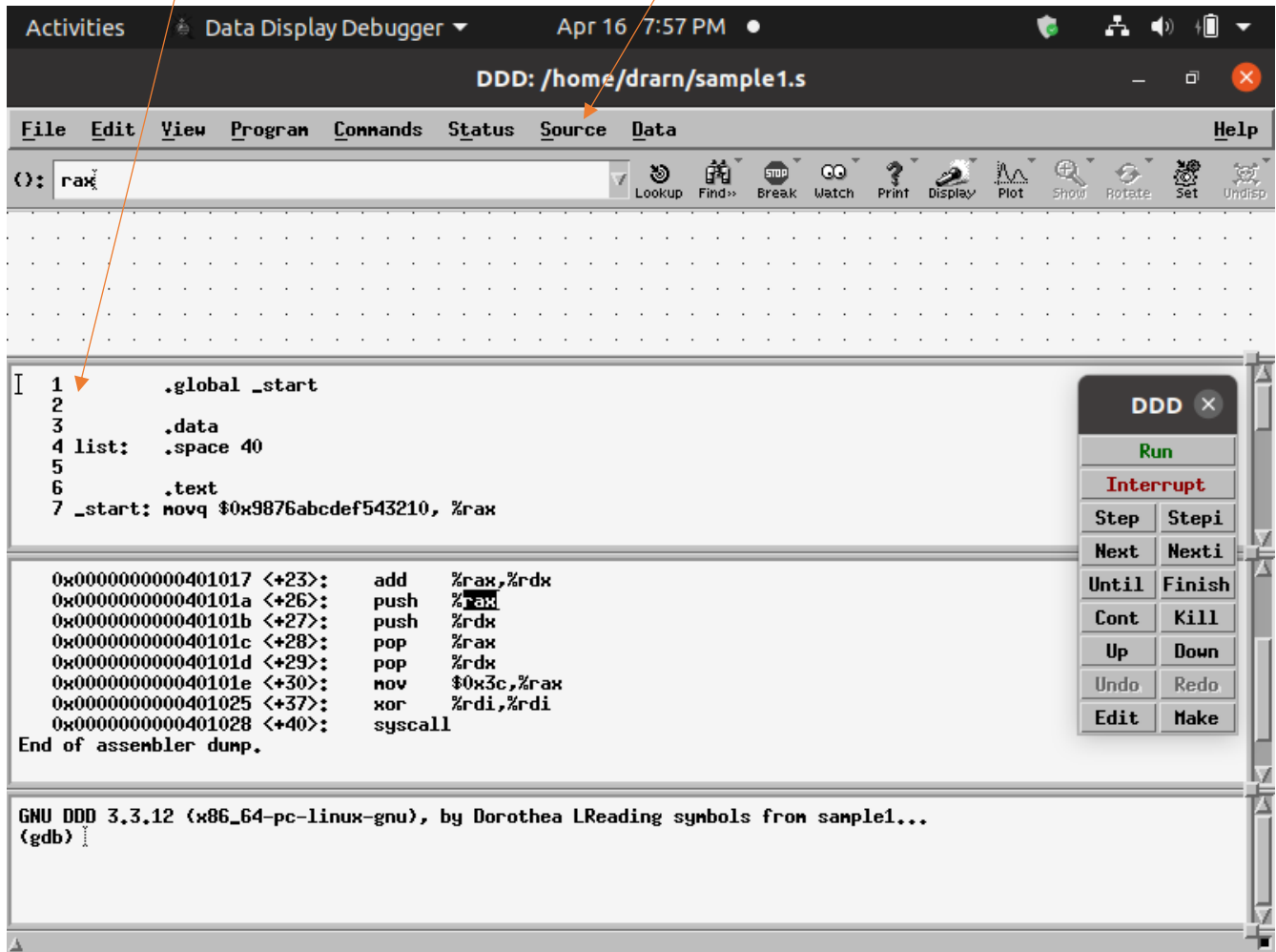
AS SHOWN BELOW – TO DISPLAY **ONLY SOURCE WINDOW**, DISABLE ALL OTHER WINDOWS BY CLICKING ON THEIR OPTIONS



TO DISPLAY **ONLY SOURCE WINDOW AND MACHINE CODE WINDOW**, SELECT THEM BY ENABLING THE OPTIONS AND DISABLE ALL OTHER WINDOWS BY CLICKING ON THEIR OPTIONS UNDER **VIEW** SUB MENU

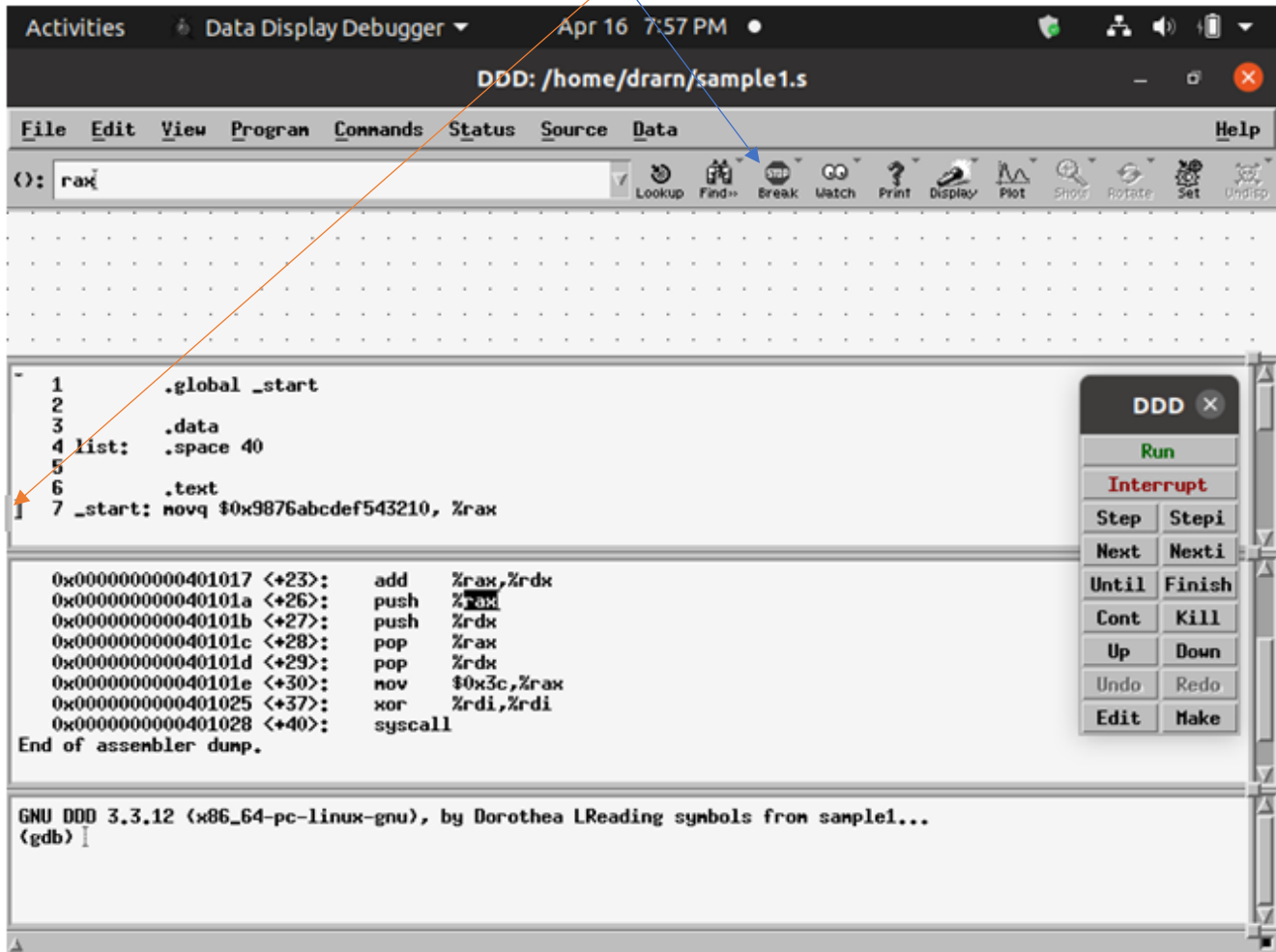


TO DISPLAY LINE NUMBERS, CLICK ON **Source** SUB MENU AND SELECT **Display line numbers** option



BEFORE START EXECUTING YOUR PROGRAM, PUT BREAKPOINT AT THE FIRST INSTRUCTION IN YOUR PROGRAM, THAT IS, INSTRUCTION AT LINE NO. 7 WITH THE LABEL `_start`:

IN ORDER TO DO THAT FIRST MOVE THE CURSOR TO LINE NO.7 AND KEEP CURSOR BEFORE LINE NO. 7 AND CLICK ON **Break** BUTTON



BREAKPOINT IS SET AT LINE NO. 7 – FIRST INSTRUCTION

The screenshot shows the Data Display Debugger (DDD) window titled "DDD: /home/drarn/sample1.s". The interface includes a menu bar (File, Edit, View, Program, Commands, Status, Source, Data, Help) and a toolbar with icons for various debugging actions. The main window is divided into several panes. The top pane shows the source code of "sample1.s" with line numbers 1 through 6 visible. Line 7 is highlighted with a red circle and the word "STOP" next to it, indicating a breakpoint. The code for line 7 is: `_start: movq $0x9876abcdef543210, %rax`. Below the source code, there is a pane showing assembly instructions with their addresses and disassembled code. The assembly instructions are: `0x0000000000401017 <+23>: add %rax,%rdx`, `0x000000000040101a <+26>: push %rax`, `0x000000000040101b <+27>: push %rdx`, `0x000000000040101c <+28>: pop %rax`, `0x000000000040101d <+29>: pop %rdx`, `0x000000000040101e <+30>: mov $0x3c,%rax`, `0x0000000000401025 <+37>: xor %rdi,%rdi`, and `0x0000000000401028 <+40>: syscall`. Below the assembly instructions, there is a pane showing the status of the debugger, including the command `(gdb) break sample1.s:7` and the message `Breakpoint 1 at 0x401000: file sample1.s, line 7.`. The bottom pane shows the command `(gdb) I`. On the right side of the window, there is a vertical toolbar with buttons for "Run", "Interrupt", "Step", "Stepi", "Next", "Nexti", "Until", "Finish", "Cont", "Kill", "Up", "Down", "Undo", "Redo", "Edit", and "Make".

```
1      .global _start
2
3      .data
4 list: .space 40
5
6      .text
7 _start: movq $0x9876abcdef543210, %rax

0x0000000000401017 <+23>:  add    %rax,%rdx
0x000000000040101a <+26>:  push   %rax
0x000000000040101b <+27>:  push   %rdx
0x000000000040101c <+28>:  pop     %rax
0x000000000040101d <+29>:  pop     %rdx
0x000000000040101e <+30>:  mov     $0x3c,%rax
0x0000000000401025 <+37>:  xor     %rdi,%rdi
0x0000000000401028 <+40>:  syscall

End of assembler dump.

GNU DDD 3.3.12 (x86_64-pc-linux-gnu), by Dorothea L
(gdb) break sample1.s:7
Breakpoint 1 at 0x401000: file sample1.s, line 7.
(gdb) I

Breakpoint 1 at 0x401000: file sample1.s, line 7.
```

THEN SET THE BREKPOINT AT THOSE INSTRUCTIONS LIKE syscall, call printf, call scanf, WHICH ARE TO BE EXECUTED USING RUN AND NOT USING SINGLE STEP

IN OUR PROGRAM, WE NEED TO SET BREAKPOINT AT LINE NO. 19, THAT IS, syscall INSTRUCTION AS SHOWN BELOW.

The screenshot shows the Data Display Debugger (DDD) window for the file `sample1.s`. The interface includes a menu bar (File, Edit, View, Program, Commands, Status, Source, Data, Help), a toolbar with various debugging actions, and a main display area. The main display area is divided into three sections: a source code view, an assembler dump, and a GDB console.

In the source code view, the following instructions are listed:

```
15      popq %rdx
16
17      movq $60, %rax
18      xorq %rdi, %rdi
19      syscall
20
```

A red circle with the word "STOP" is placed next to line 19, indicating that the program has halted at this instruction. An orange arrow points from the text "IN OUR PROGRAM, WE NEED TO SET BREAKPOINT AT LINE NO. 19, THAT IS, syscall INSTRUCTION AS SHOWN BELOW." to this "STOP" marker.

The assembler dump section shows the corresponding machine code instructions:

```
0x0000000000401017 <+23>: add    %rax,%rdx
0x000000000040101a <+26>: push  %rax
0x000000000040101b <+27>: push  %rdx
0x000000000040101c <+28>: pop   %rax
0x000000000040101d <+29>: pop   %rdx
0x000000000040101e <+30>: mov   $0x3c,%rax
0x000000000040101f <+31>: xor   %rdi,%rdi
0x0000000000401020 <+32>: syscall
0x0000000000401021 <+33>:
0x0000000000401022 <+34>:
0x0000000000401023 <+35>:
0x0000000000401024 <+36>:
0x0000000000401025 <+37>:
0x0000000000401026 <+38>:
0x0000000000401027 <+39>:
0x0000000000401028 <+40>: syscall
End of assembler dump.
```

The GDB console at the bottom shows the following commands and output:

```
(gdb) break sample1.s:7
Breakpoint 1 at 0x401000: file sample1.s, line 7.
(gdb) break sample1.s:19
Breakpoint 2 at 0x401028: file sample1.s, line 19.
(gdb) I
```

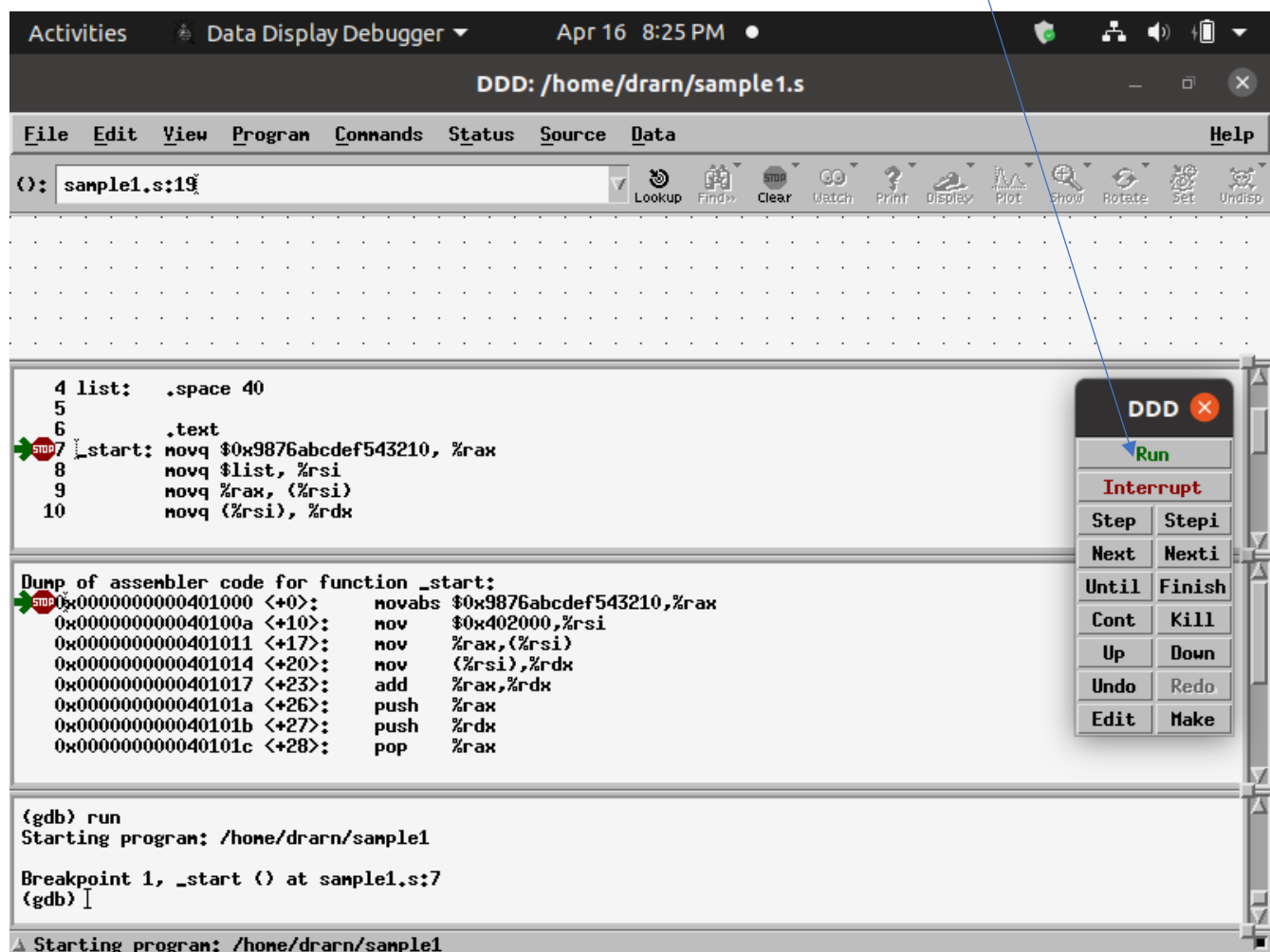
A status bar at the bottom of the window displays the message: `Breakpoint 2 at 0x401028: file sample1.s, line 19.`

NOW EXECUTE THE PROGRAM BY CLICKING **Run** IN THE COMMAND WINDOW

SINCE THERE IS BREAKPOINT AT FIRST INSTRUCTION, EXECUTION WILL STOP OR PAUSE AT FIRST INSTRUCTION

ALL GENERAL PURPOSE REGISTERS ARE CLEARED AND MEMORY LOCATIONS ARE CLEARED

%rip points to CODE SEGMENT AND %rsp POINTS TO TOP OF THE STACK



The screenshot shows the Data Display Debugger (DDD) window. The title bar indicates the file path is `DDD: /home/drarn/sample1.s`. The menu bar includes `File`, `Edit`, `View`, `Program`, `Commands`, `Status`, `Source`, `Data`, and `Help`. The toolbar contains icons for `Lookup`, `Find`, `Clear`, `Watch`, `Print`, `Display`, `Plot`, `Show`, `Rotate`, `Set`, and `Undisp`. The main window displays assembly code for the `_start` function, with a red stop icon and a green arrow indicating a breakpoint at line 7. The code is as follows:

```
4 list: .space 40
5
6 .text
7 _start: movq $0x9876abcdef543210, %rax
8        movq $list, %rsi
9        movq %rax, (%rsi)
10       movq (%rsi), %rdx
```

Below the assembly code, a dump of the assembler code for the `_start` function is shown, with a red stop icon and a green arrow indicating a breakpoint at the first instruction:

```
Dump of assembler code for function _start:
0x0000000000401000 <+0>:  movabs $0x9876abcdef543210,%rax
0x000000000040100a <+10>:  mov  $0x402000,%rsi
0x0000000000401011 <+17>:  mov  %rax,(%rsi)
0x0000000000401014 <+20>:  mov  (%rsi),%rdx
0x0000000000401017 <+23>:  add  %rax,%rdx
0x000000000040101a <+26>:  push %rax
0x000000000040101b <+27>:  push %rdx
0x000000000040101c <+28>:  pop  %rax
```

On the right side of the window, a control panel is visible with the following buttons:

- `Run` (highlighted with a blue arrow)
- `Interrupt`
- `Step` / `Stepi`
- `Next` / `Nexti`
- `Until` / `Finish`
- `Cont` / `Kill`
- `Up` / `Down`
- `Undo` / `Redo`
- `Edit` / `Make`

The command window at the bottom shows the following text:

```
(gdb) run
Starting program: /home/drarn/sample1

Breakpoint 1, _start () at sample1.s:7
(gdb) |
```

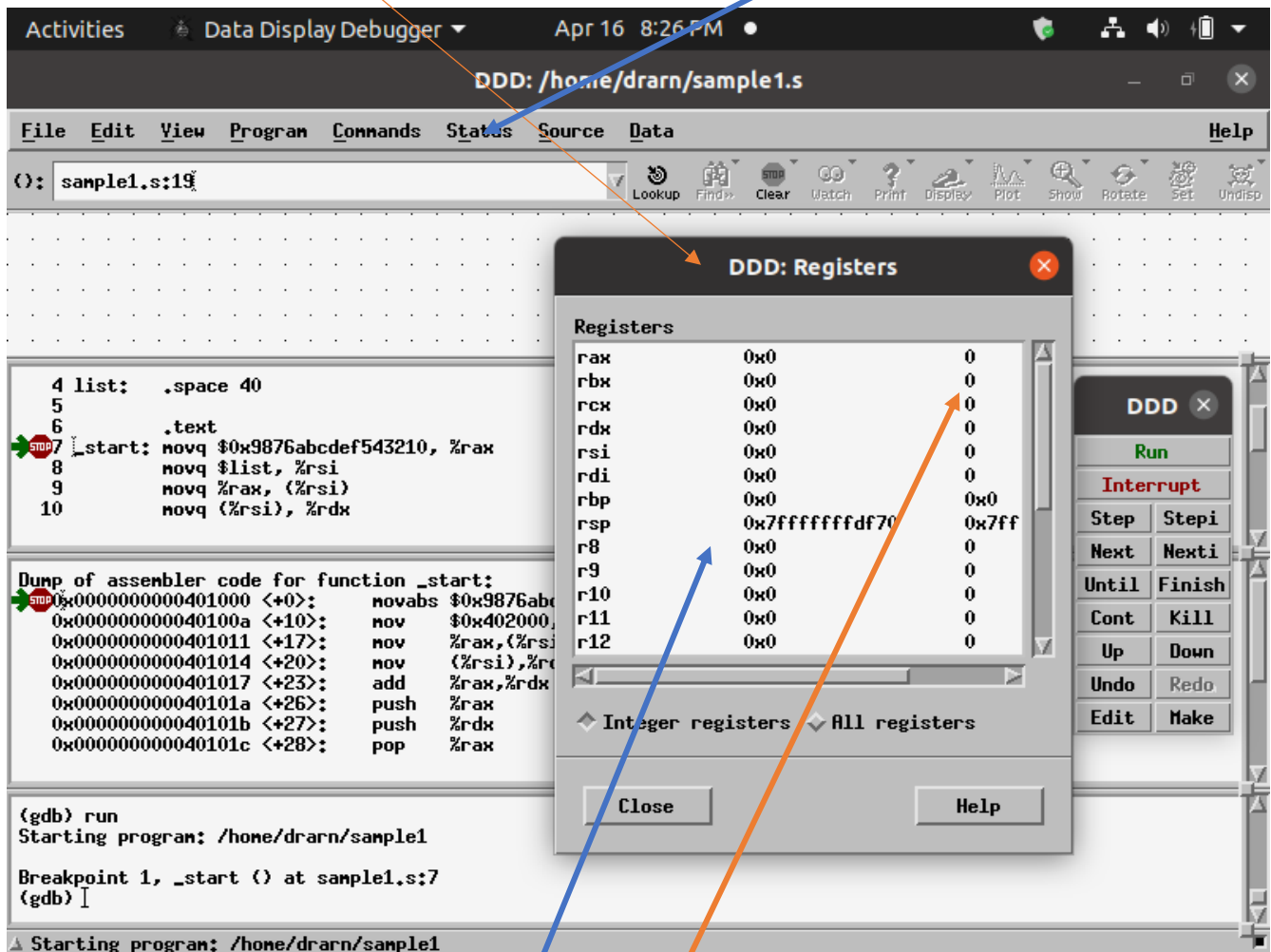
The status bar at the bottom indicates the program is starting: `Starting program: /home/drarn/sample1`.

INITIALLY ALL GENERAL PURPOSE REGISTERS ARE CLEARED AND
MEMORY LOCATIONS ARE CLEARED

%rip points to CODE SEGMENT AND %rsp POINTS TO TOP OF THE STACK

IN ORDER TO DISPLAY THE REGISTER WINDOW, CLICK OPTION **Status**
AND SELECT THE SUB OPTION **Registers** UNDER THAT MENU

NOW REGISTER WINDOW IS POPPED UP. YOU CAN VIEW THE CONTENTS
OF ALL THE REGISTERS IN HEX AND DECIMAL IN MOST OF THE CASES.



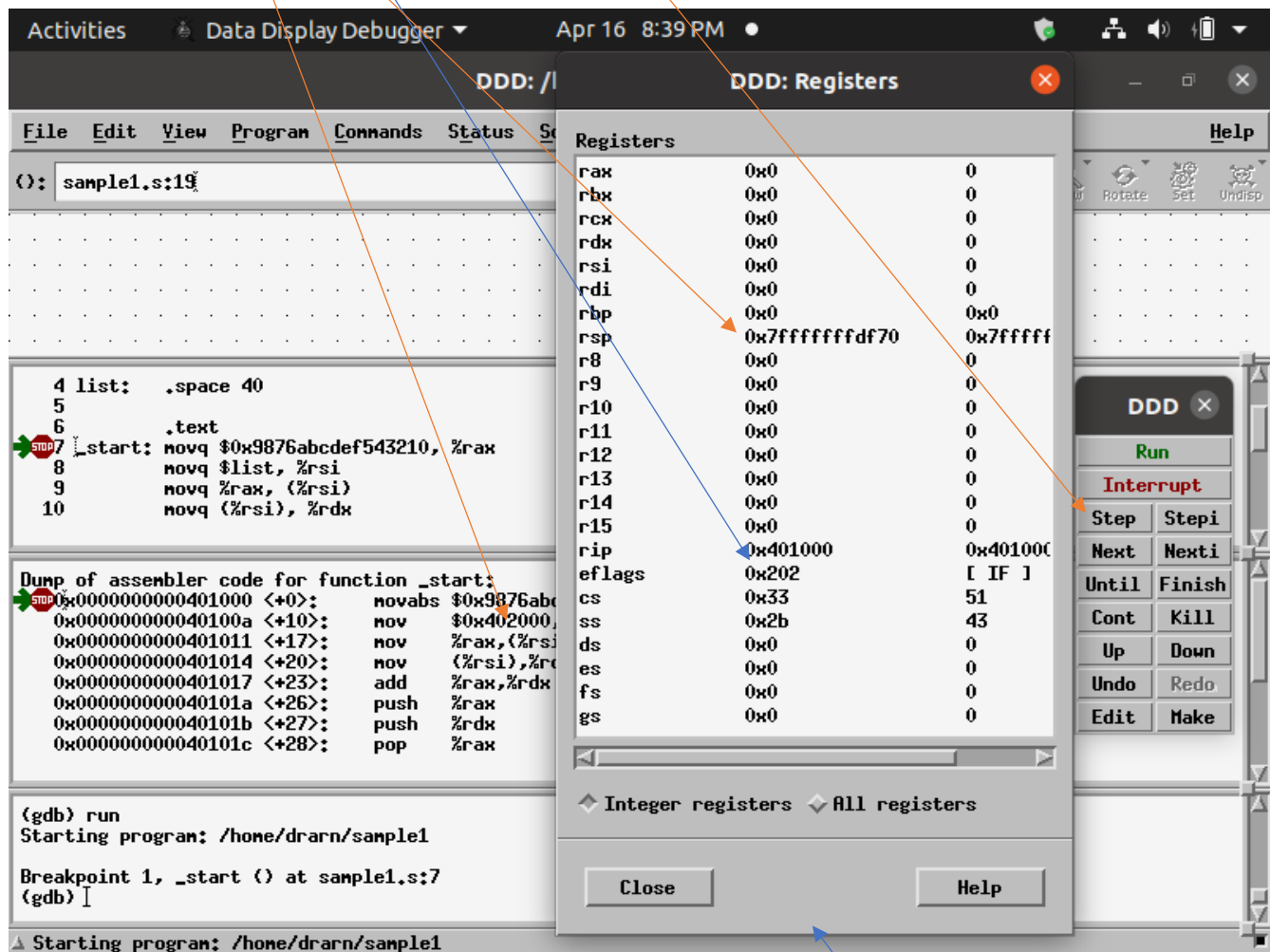
ALL REGISTER CONTENTS ARE SHOWN IN HEX
FOR GENERAL PURPOSE REGISTERS, REGISTER CONTENTS
ARE ALSO SHOWN IN DECIMAL

%rip ← 0x401000 ;START OF CODE SEGMENT

list ← 0x402000 ;START OF DATA SEGMENT

%rsp ← 0x7fffffffdf70 ; TOP OF THE STACK

CLICK step to execute first Instruction at line no.7



ENLARGE THE REGISTER WINDOW BY PULLING IT DOWN

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FIRST INSTRUCTION

movq \$0x9876abcdef543210, %rax

%rip ← 0x40100a ; ADDRESS OF SECOND INSTRUCTION

The screenshot shows the Data Display Debugger (DDD) interface. The assembly window on the left displays the following code:

```
5  
6 .text  
7 _start: movq $0x9876abcdef543210, %rax  
8         movq $list, %rsi  
9         movq %rax, (%rsi)  
10        movq (%rsi), %rdx  
11        addq %rax, %rdx
```

The registers window on the right shows the state of the registers:

Register	Value	Comment
rax	0x9876abcdef543210	-7460586831720730096
rbx	0x0	0
rcx	0x0	0
rdx	0x0	0
rsi	0x0	0
rdi	0x0	0
rbp	0x0	0x0
rsp	0x7fffffffdf70	0x7fffffffdf70
r8	0x0	0
r9	0x0	0
r10	0x0	0
r11	0x0	0
r12	0x0	0
r13	0x0	0
r14	0x0	0
r15	0x0	0
rip	0x40100a	0x40100a <_start+10>
eflags	0x202	[IF]
cs	0x33	51
ss	0x2b	43
ds	0x0	0
es	0x0	0
fs	0x0	0
gs	0x0	0

The control panel on the right shows the 'Step' button highlighted. The status bar at the bottom indicates 'Step'.

NOW CLICK **step** to execute second Instruction at line no.8

movq \$list, %rsi

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF SECOND INSTRUCTION

movq \$list, %rsi

list ← 0x402000 ; START OF DATA BLOCK

%rip ← 0x401011 ; ADDRESS OF THIRD INSTRUCTION

The screenshot shows the Data Display Debugger (DDD) interface. The main window is titled "DDD: Registers". On the left, there is a list of registers with their values and decimal equivalents. The register %rsi is highlighted with a red arrow pointing to its value 0x402000. Below the register list, there is a section for "Integer registers" and "All registers". At the bottom, there are buttons for "Close" and "Help".

Registers

Register	Value	Decimal
rax	0x9876abcdef543210	-7460586831720730096
rbx	0x0	0
rcx	0x0	0
rdx	0x0	0
rsi	0x402000	4202496
rdi	0x0	0
rbp	0x0	0x0
rsp	0x7fffffffdf70	0x7fffffffdf70
r8	0x0	0
r9	0x0	0
r10	0x0	0
r11	0x0	0
r12	0x0	0
r13	0x0	0
r14	0x0	0
r15	0x0	0
rip	0x401011	0x401011 <_start+17>
eflags	0x202	[IF]
cs	0x33	51
ss	0x2b	43
ds	0x0	0
es	0x0	0
fs	0x0	0
gs	0x0	0

Integer registers All registers

Close Help

Assembly code:

```
6      .text
7  _start: movq $0x9876abcdef543210, %rax
8      movq $list, %rsi
9      movq %rax, (%rsi)
10     movq (%rsi), %rdx
11     addq %rax, %rdx
12     pushq %rax
```

Dump of assembler code for function _start:

```
0x0000000000401000 <+0>: no
0x000000000040100a <+10>: no
0x0000000000401011 <+17>: no
0x0000000000401014 <+20>: no
0x0000000000401017 <+23>: ad
0x000000000040101a <+26>: pu
0x000000000040101b <+27>: pu
0x000000000040101c <+28>: po
```

Breakpoint 1, _start () at sample1.c:7

(gdb) step

(gdb) step

(gdb) I

Step

NOW CLICK step to execute THIRD Instruction at line no.9
movq %rax, (%rsi)

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF THIRD INSTRUCTION
movq %rax, (%rsi)
CONTENTS OF %rax REGISTER ARE WRITTEN INTO MEMORY STARTING FROM 0x402000

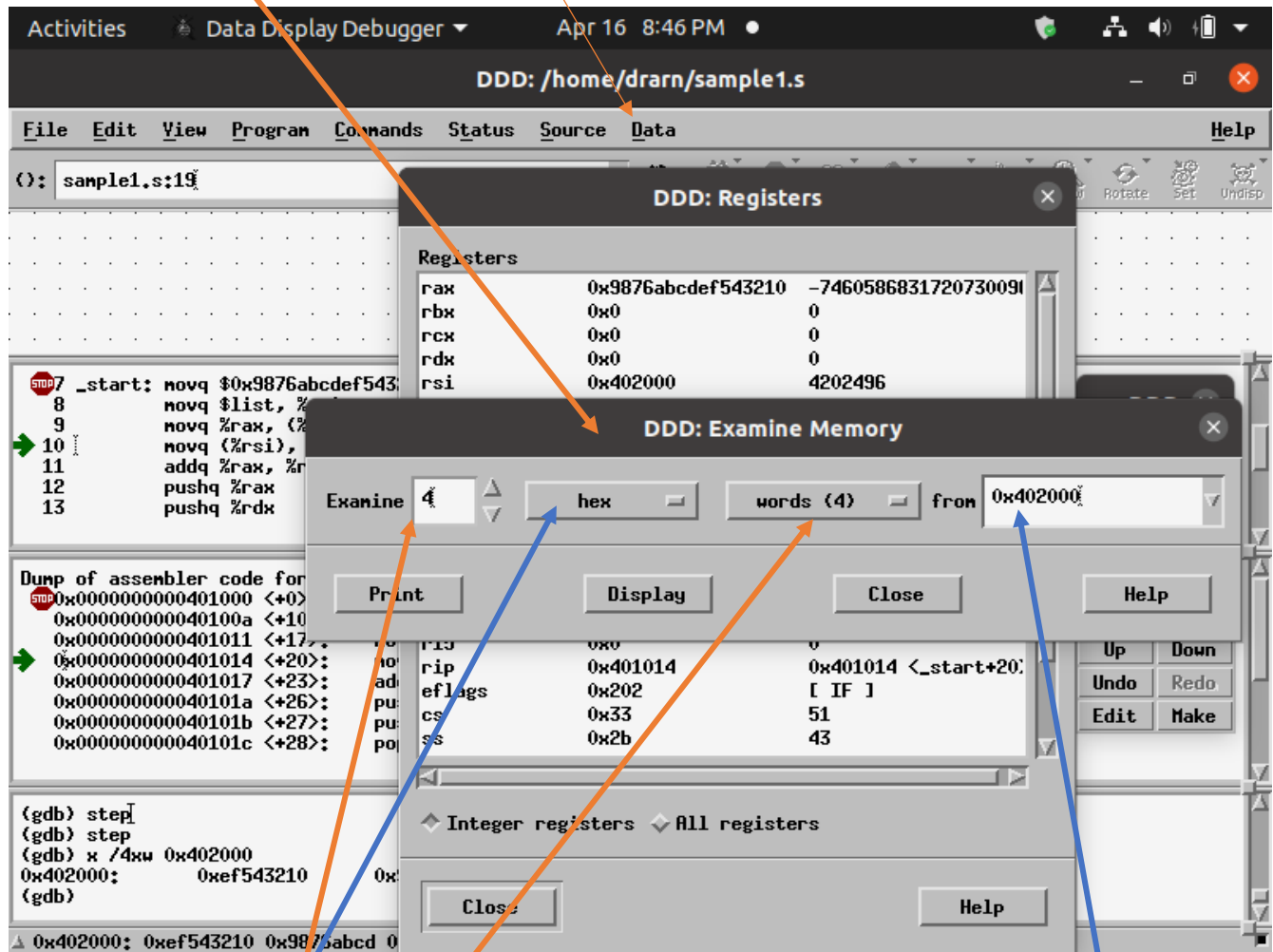
%rip ← 0x401014 ; ADDRESS OF FOURTH INSTRUCTION

The screenshot shows the Data Display Debugger (DDD) interface. The main window displays the assembly code for a function, with the third instruction highlighted: `movq (%rsi), %rdx` at address `0x401014`. The register window on the right shows the state of the registers after the execution of the third instruction. The `rip` register is at `0x401014`, and the `rsi` register is at `0x402000`. The `rax` register contains the value `0x9876abcdef543210`. The instruction list on the left shows the sequence of instructions, with the third instruction being the one that was executed.

Register	Value	Comment
rax	0x9876abcdef543210	-7460586831720730096
rbx	0x0	0
rcx	0x0	0
rdx	0x0	0
rsi	0x402000	4202496
rdi	0x0	0
rbp	0x0	0x0
rsp	0x7fffffffdf70	0x7fffffffdf70
r8	0x0	0
r9	0x0	0
r10	0x0	0
r11	0x0	0
r12	0x0	0
r13	0x0	0
r14	0x0	0
r15	0x0	0
rip	0x401014	0x401014 <_start+20>
eflags	0x202	[IF]
cs	0x33	51
ss	0x2b	43
ds	0x0	0
es	0x0	0
fs	0x0	0
gs	0x0	0

LET US NOW LOOK AT MEMORY STARTING AT 0x402000 where contents of %rax are written to memory area pointed to by %rsi = 0x402000

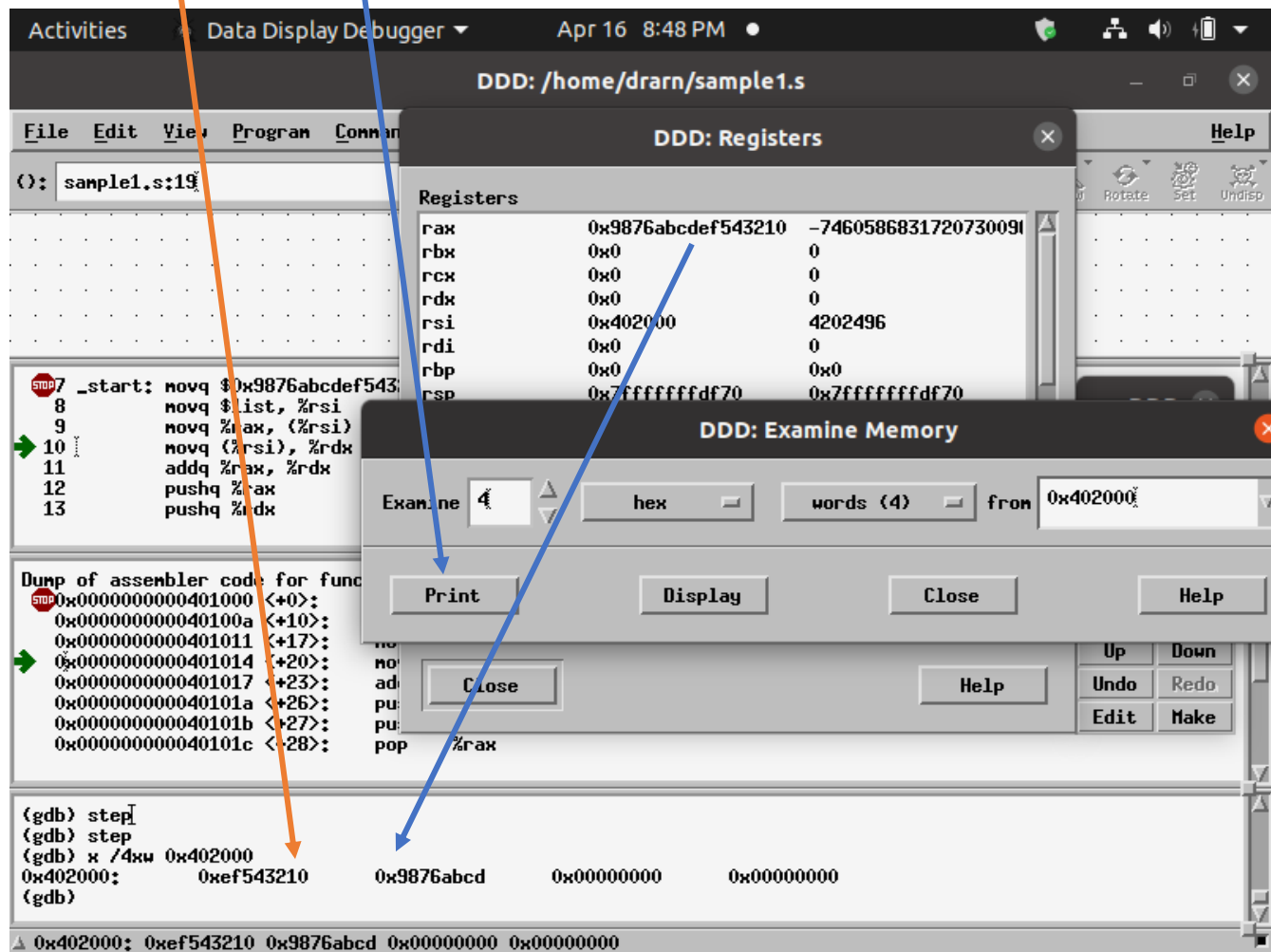
TO DO THIS, CLICK ON **Data** OPTION IN MENU AND SELECT SUB OPTION **Memory** UNDER THAT SUB MENU
NOW **Examine Memory** WINDOW WILL BE POPPED UP



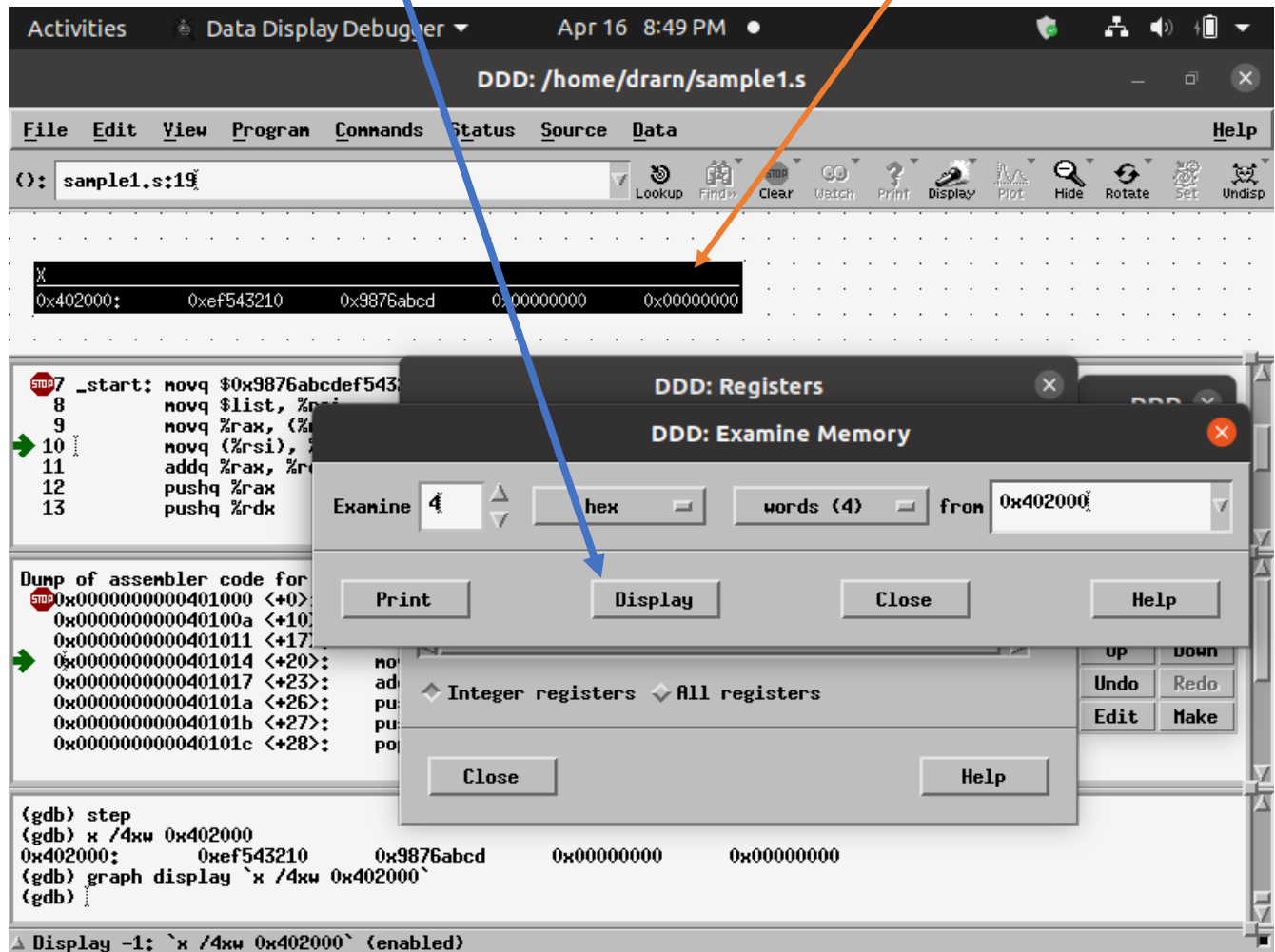
TO EXAMINE THE CONTENTS OF MEMORY STARTING FROM 0x402000,

1. ENTER STARTING ADDRESS 0x402000 in from text box as shown above
2. DATA TO BE DISPLAYED TOGETHER EITHER IN BYTE OR HALFWORD(2 BYTES) OR WORD(4 BYTES) FORMAT – SELECT words(4) as shown above
3. DATA display FORMAT – decimal, Octal, hex - SELECT hex as shown above
4. NUMBER OF WORDS TO BE EXAMINED STARTING FROM ADDRESS 0x402000 – ENTER 4 IN THE TEXT BOX AFTER Examine TEXT BOX as shown above

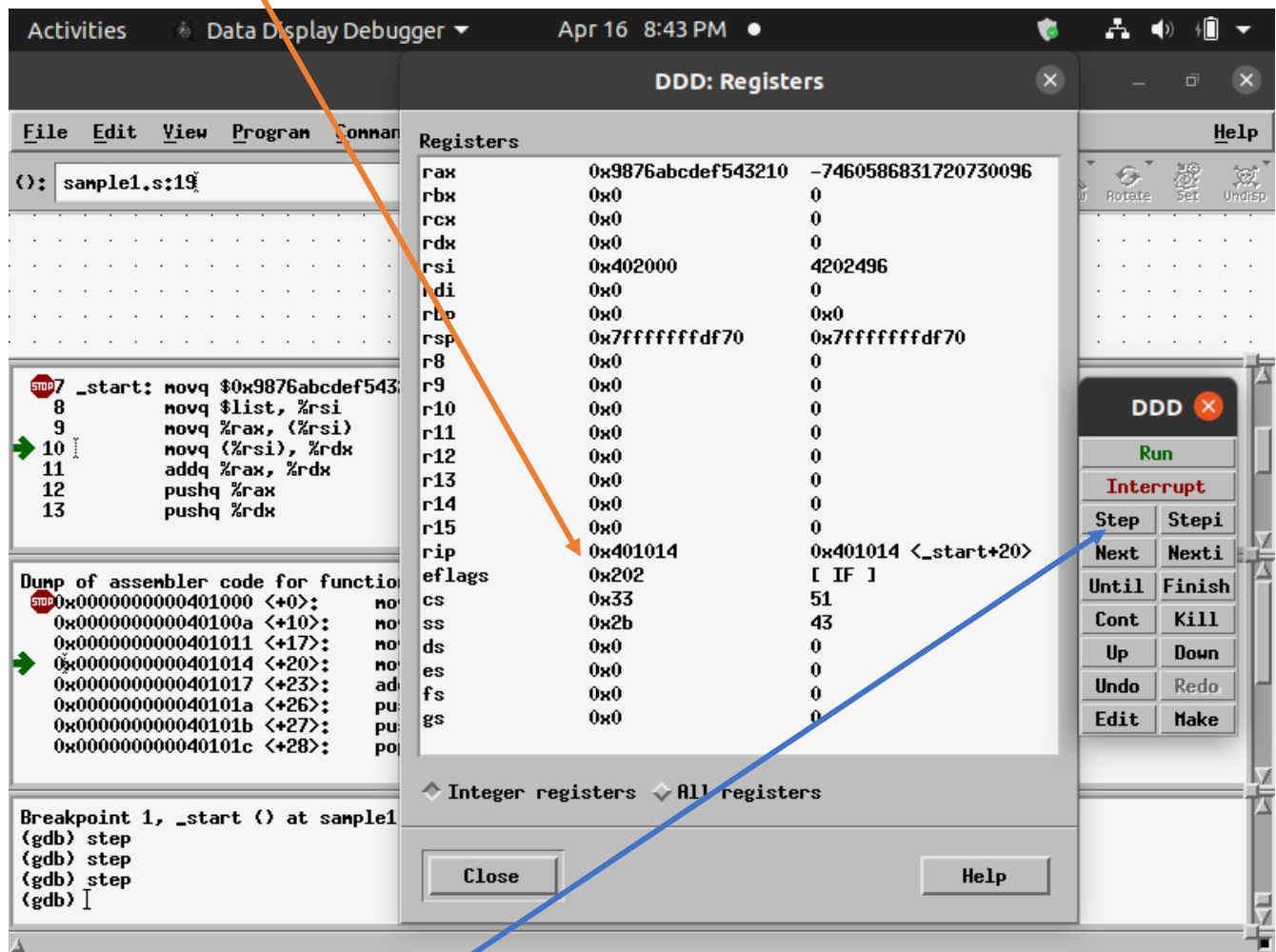
NOW CLICK **Print** BUTTON TO PRINT 4 WORD MEMORY CONTENTS STARTING FROM ADDRESS 0x402000 IN GDB CONSOLE WINDOW AS SHOWN BELOW -



NOW CLICK **Display** BUTTON TO PRINT 4 WORD MEMORY CONTENTS STARTING FROM ADDRESS 0x402000 IN DATA WINDOW AS SHOWN BELOW -



%rip ← 0x401014 ; ADDRESS OF FOURTH INSTRUCTION

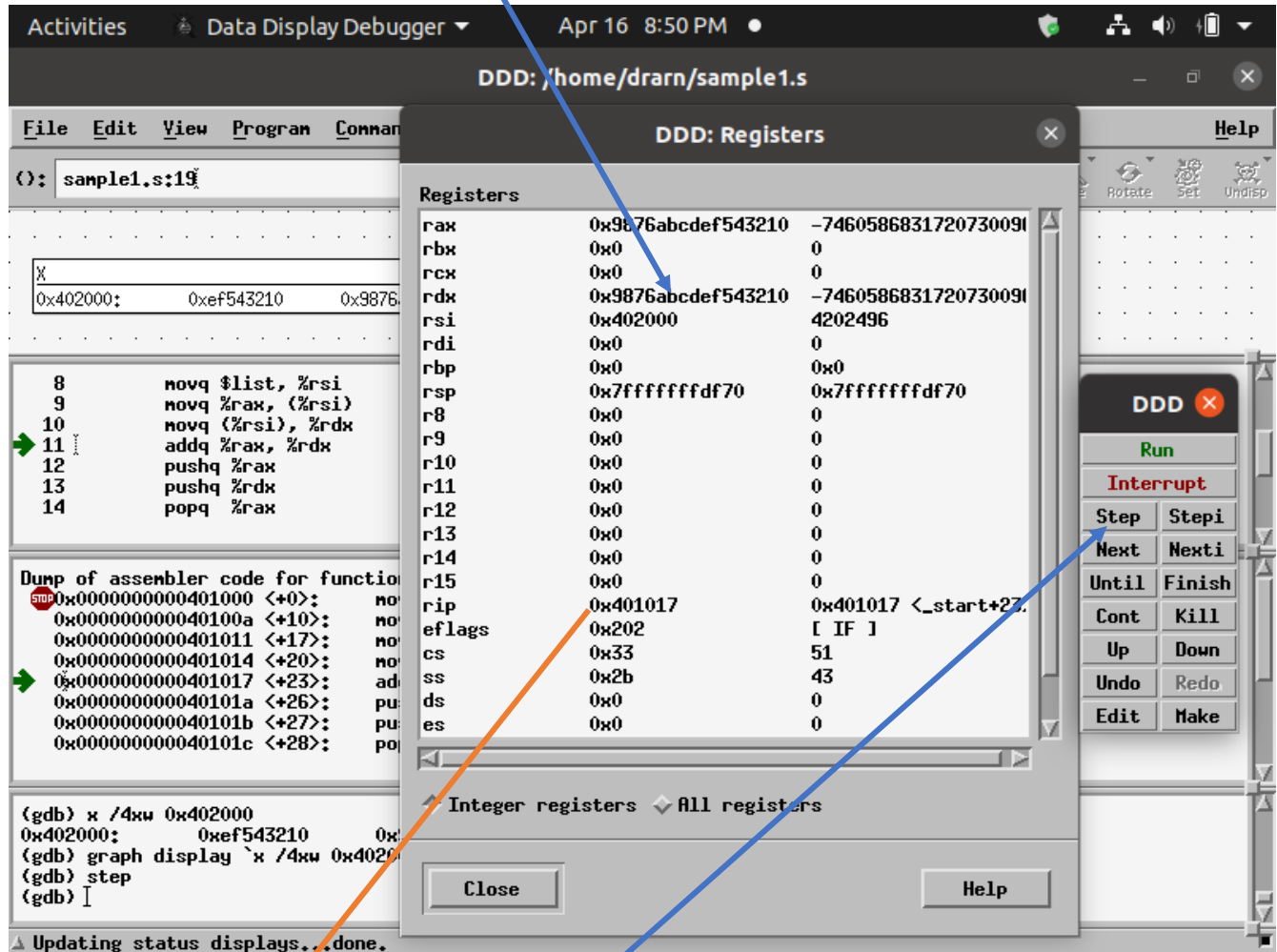


NOW CLICK `step` to execute FOURTH Instruction at line no. 10
`movq (%rsi), %rdx`

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FOURTH INSTRUCTION

movq (%rsi), %rdx

8 BYTES FROM MEMORY STARTING FROM 0x402000 POINTED TO BY %rsi ARE MOVED TO REGISTER %rdx

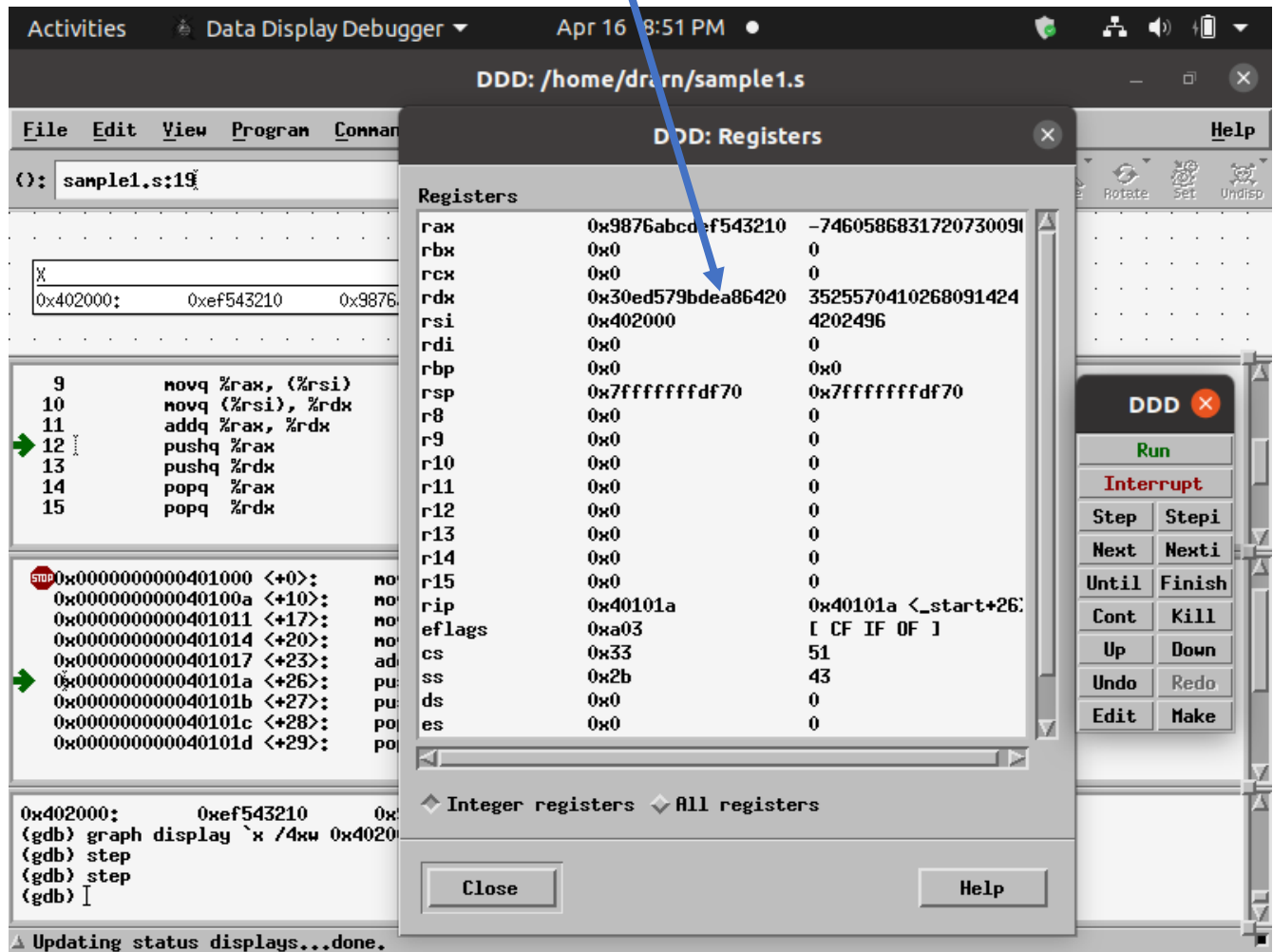


%rip ← 0x401017 ; ADDRESS OF FIFTH INSTRUCTION

NOW CLICK **step** to execute FIFTH Instruction at line no. 11
addq %rax, %rdx

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FIFTH INSTRUCTION **addq %rax, %rdx**

LOOK AT CONTENTS OF REGISTER %rdx



%rip ← 0x40101a ; ADDRESS OF SIXTH INSTRUCTION

NOW CLICK step to execute SIXTH Instruction at line no. 12
pushq %rax

%rsp is decremented by 8

%rsp = 0x7fffffffdf68

Contents of %rax are written onto the stack

Activities Data Display Debugger Apr 16 8:52 PM

DDD: /home/drarn/sample1.s

File Edit View Program Command

(): sample1.s:19

0x402000: 0xef543210 0x9876

10 movq (%rsi), %rdx
11 addq %rax, %rdx
12 pushq %rax
13 pushq %rdx
14 popq %rax
15 popq %rdx
16

0x000000000040100a <+10>: no
0x0000000000401011 <+17>: no
0x0000000000401014 <+20>: no
0x0000000000401017 <+23>: ad
0x000000000040101a <+26>: pu
0x000000000040101b <+27>: pu
0x000000000040101c <+28>: po
0x000000000040101d <+29>: po
0x000000000040101e <+30>: no

(gdb) step
(gdb) step
(gdb) step
_start () at sample1.s:13
(gdb) I

Updating status displays...done.

Registers

rax	0x9876abcdef543210	-7460586831720730091
rbx	0x0	0
rcx	0x0	0
rdx	0x30ed579bdea86420	3525570410268091424
rsi	0x402000	4202496
rdi	0x0	0
rbp	0x0	0x0
rsp	0x7fffffffdf68	0x7fffffffdf68
r8	0x0	0
r9	0x0	0
r10	0x0	0
r11	0x0	0
r12	0x0	0
r13	0x0	0
r14	0x0	0
r15	0x0	0
rip	0x40101b	0x40101b <_start+27>
eflags	0xa03	[CF IF OF]
cs	0x33	51
ss	0x2b	43
ds	0x0	0
es	0x0	0

Integer registers All registers

Close Help

DDD X

Run

Interrupt

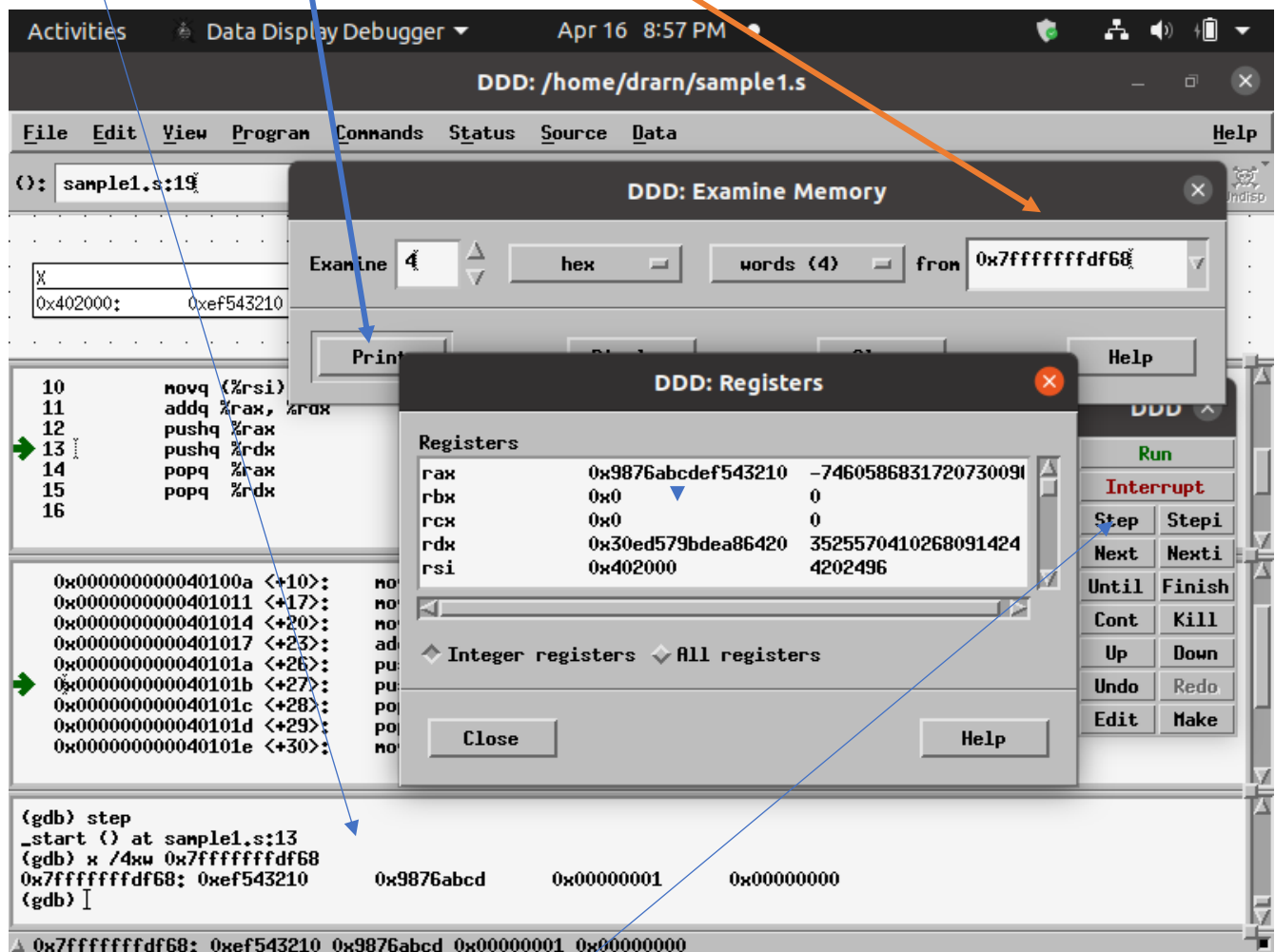
Step Step
Next Next
Until Finish
Cont Kill
Up Down
Undo Redo
Edit Make

%rip ← 0x40101b ; ADDRESS OF SEVENTH INSTRUCTION

pushq %rdx

LET US NOW LOOK AT THE CONTENTS OF THE STACK AT 0x7fffffffdf68

NOW CLICK **Print** BUTTON TO PRINT 4 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf68 IN GDB CONSOLE WINDOW AS SHOWN BELOW -



`%rip` ← `0x40101b` ; ADDRESS OF SEVENTH INSTRUCTION

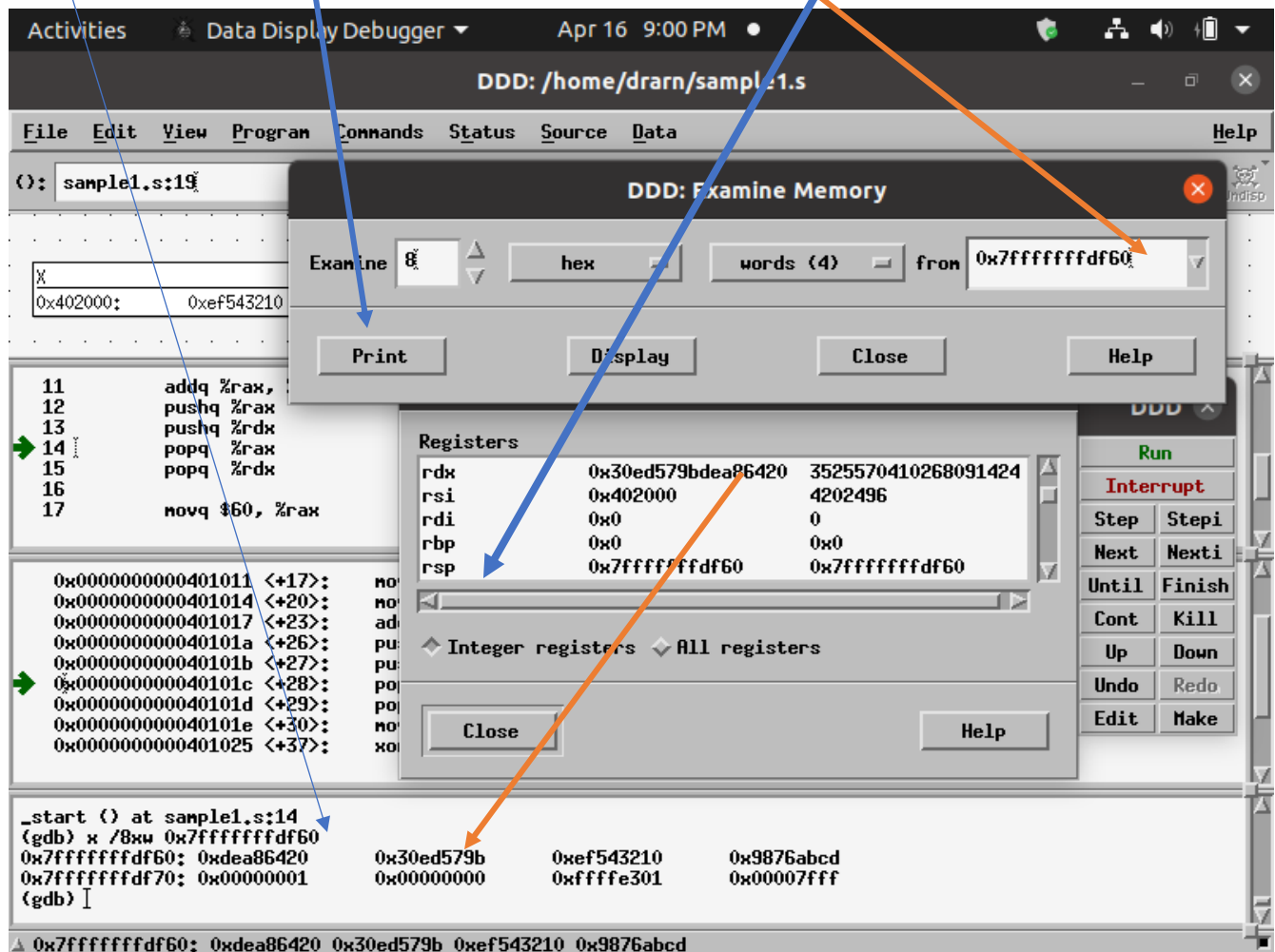
NOW CLICK **step** to execute SEVENTH Instruction at line no. 13
`pushq %rdx`

%rsp is decremented by 8

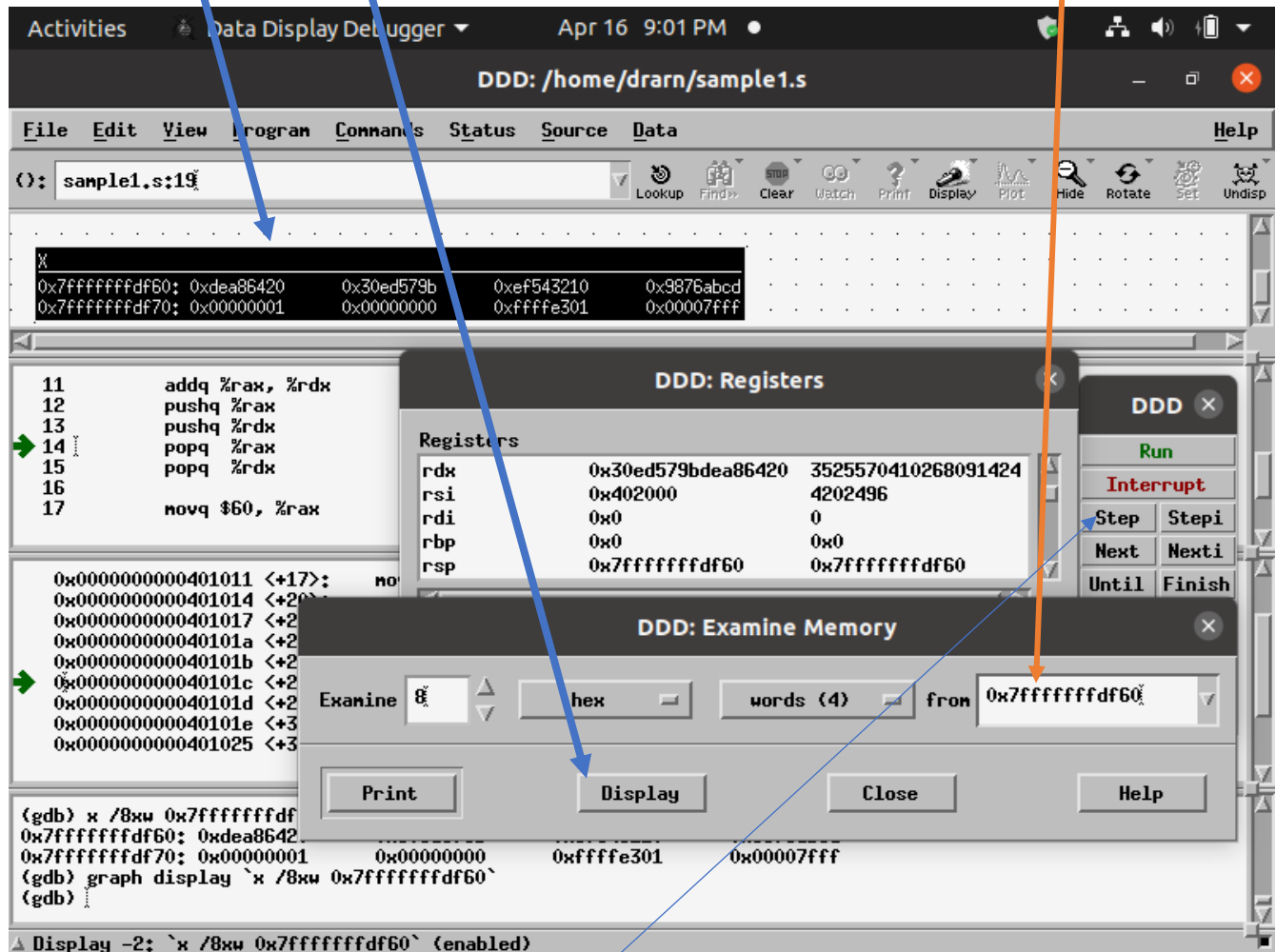
%rsp = 0x7fffffffdf60

LET US NOW LOOK AT THE CONTENTS (8 WORDS) OF THE STACK AT 0x7fffffffdf60

NOW CLICK **Print** BUTTON TO PRINT 8 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf60 IN GDB CONSOLE WINDOW AS SHOWN BELOW



NOW CLICK **Display** BUTTON TO PRINT 8 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf60 IN DATA WINDOW AS SHOWN BELOW -

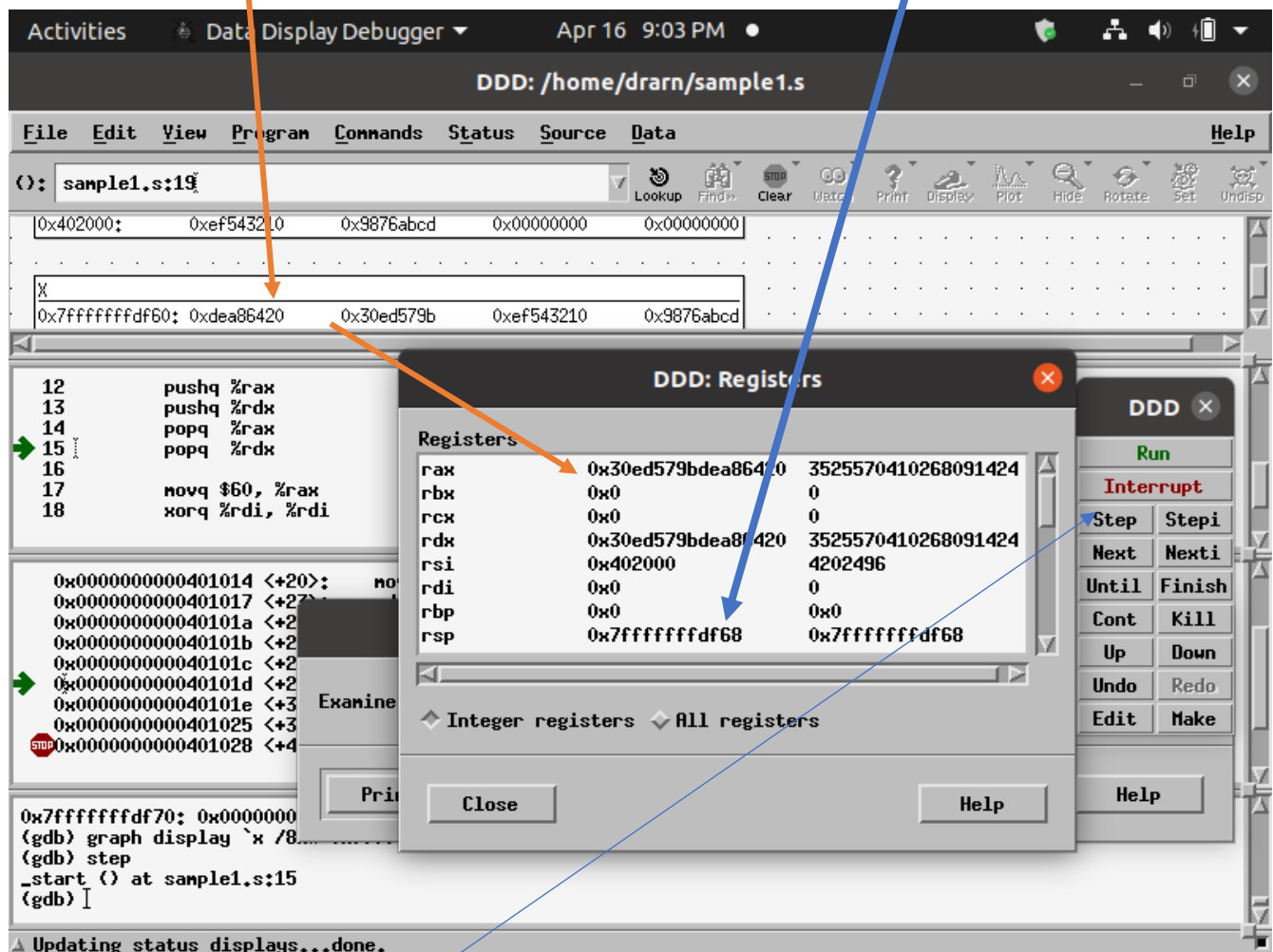


%rip ← 0x40101c ; ADDRESS OF EIGHTH INSTRUCTION

NOW CLICK **step** to execute EIGHTH Instruction at line no. 14
popq %rax

Contents (8 bytes) of stack memory area pointed to by %rsp = 0x7fffffffdf60 is read into %rax register

After that %rsp is incremented by 8 %rsp = 0x7fffffffdf68



%rip ← 0x40101d ; ADDRESS OF NINTH INSTRUCTION

NOW CLICK step to execute NINTH Instruction at line no. 15
popq %rdx

Contents (8 bytes) of stack memory area pointed to by %rsp = 0x7fffffffdf68 is read into %rdx register

After that %rsp is incremented by 8 %rsp = 0x7fffffffdf70

The screenshot shows the Data Display Debugger (DDD) interface. The main window displays assembly code for `sample1.s`. The assembly code is as follows:

```
14      popq %rax
15      popq %rdx
16
17      movq $60, %rax
18      xorq %rdi, %rdi
19      syscall
20
```

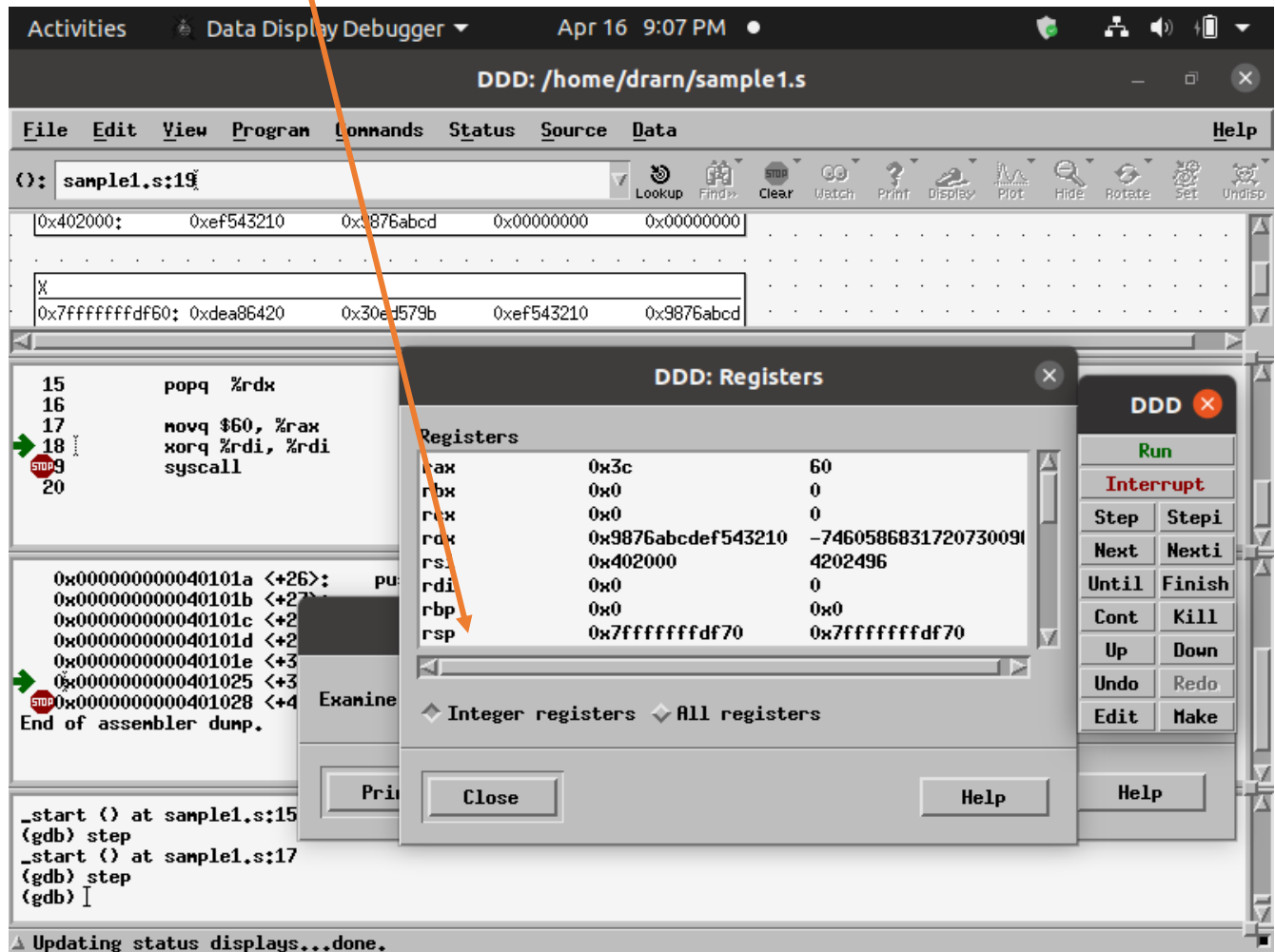
The assembly code is shown in the left pane. The right pane shows the memory dump. The memory address `0x7fffffffdf60` is highlighted, and its contents are `0xdea86420 0x30ed579b 0xef543210 0x9876abcd`. The `Registers` window is open, showing the current state of the registers. The `rdx` register is highlighted, and its value is `0x9876abcd543210`. The `rsp` register is also highlighted, and its value is `0x7fffffffdf70`. The `Registers` window is titled "DDD: Registers" and contains a table of registers and their values.

Register	Value (Hex)	Value (Dec)
rax	0x30ed579bdea86420	3525570410268091424
rbx	0x0	0
rcx	0x0	0
rdx	0x9876abcd543210	-7460586831720730096
rsi	0x402000	4202496
rdi	0x0	0
rbp	0x0	0
rsp	0x7fffffffdf70	0x7fffffffdf70

The `Registers` window also includes a section for "Integer registers" and "All registers". The `Integer registers` section is currently selected. The `Registers` window has a "Close" button and a "Help" button. The main window has a "File" menu, an "Edit" menu, a "View" menu, a "Program" menu, a "Commands" menu, a "Status" menu, a "Source" menu, and a "Data" menu. The main window also has a toolbar with buttons for "Look up", "Find", "Clear", "Watch", "Print", "Display", "Plot", "Hide", "Rotate", "Set", and "Undisp".

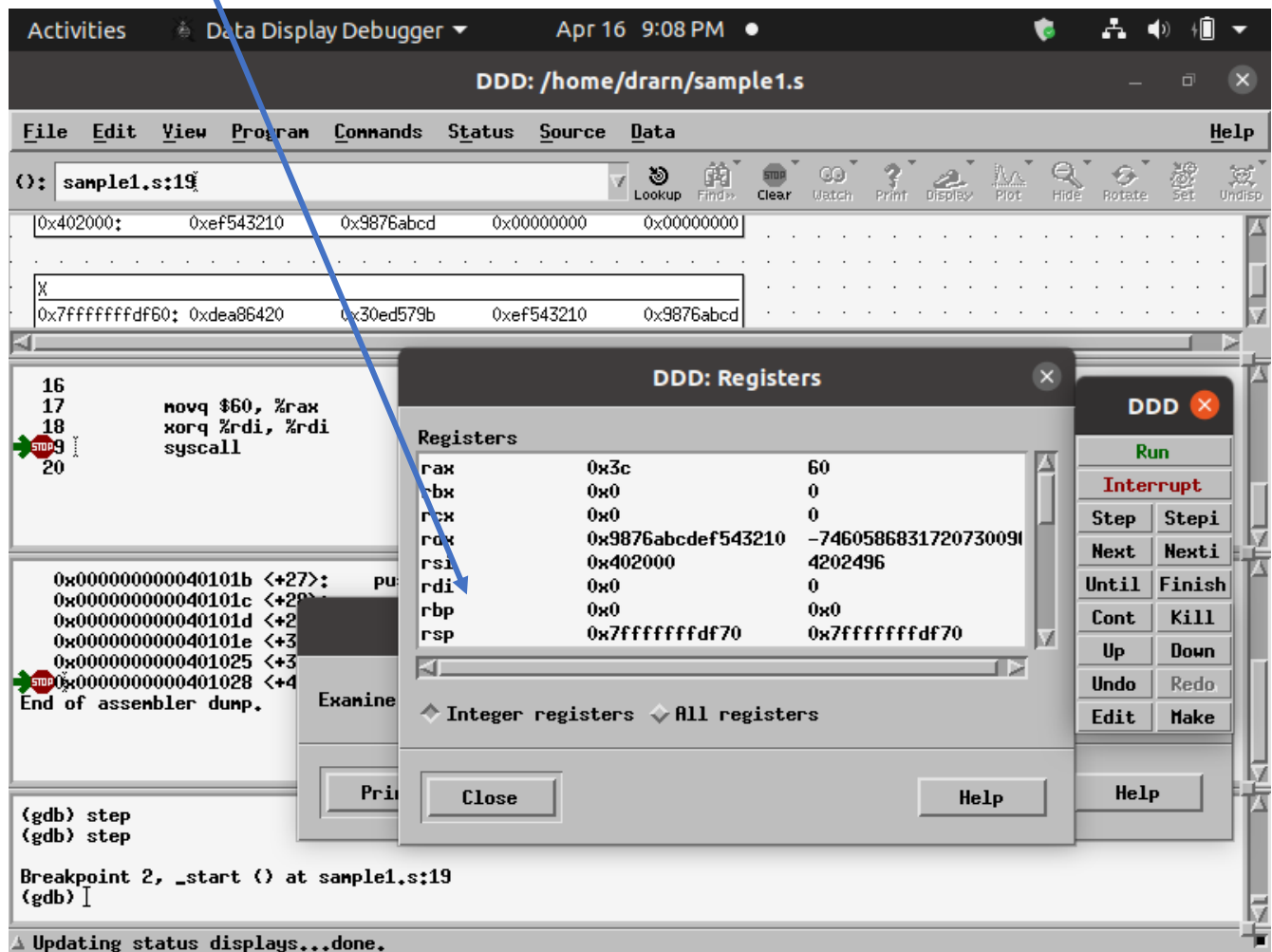
%rip ← 0x40101e ; ADDRESS OF TENTH INSTRUCTION

NOW CLICK step to execute TENTH Instruction at line no. 17
movq \$60, %rax



Next %rip ← 0x401025 ; ADDRESS OF ELEVENTH INSTRUCTION

NOW CLICK step to execute ELEVENTH Instruction at line no. 18
xorq %rdi, %rdi

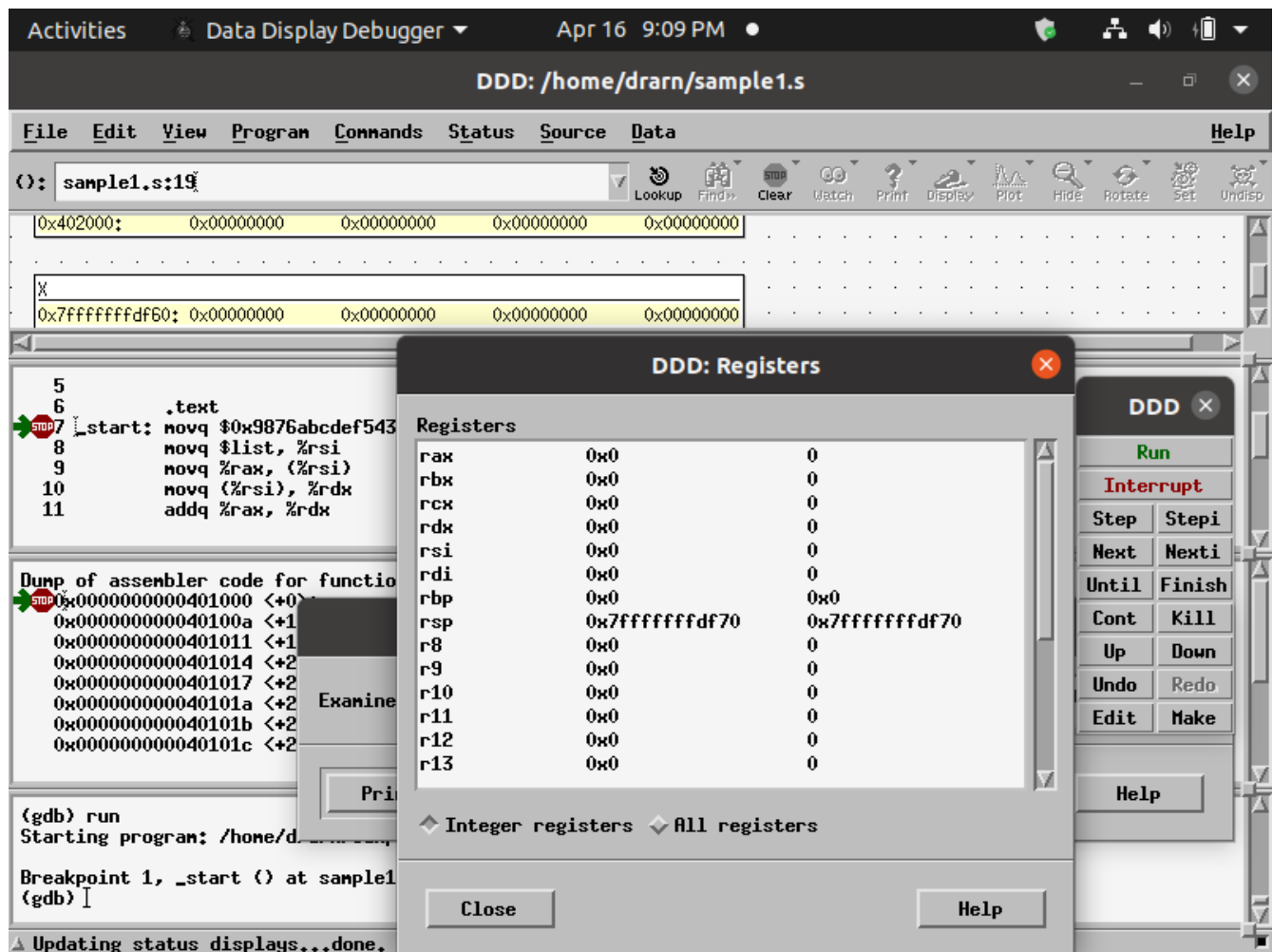


Next %rip ← 0x401028 ; ADDRESS OF TWELVETH INSTRUCTION

NOW CLICK run to execute TWELVETH Instruction at line no. 19
syscall

NOW LOOK AT THE REGISTERS AND MEMORY LOCATIONS AFTER THE EXECUTION OF syscall

ALL ARE INITIALIZED TO THEIR ORIGINAL VALUES AND %rip IS NOW POINTING TO THE FIRST INSTRUCTION



**END OF DDD DEBUGGER TUTORIAL SESSION
WISH YOU ALL THE BEST**
